VUE FINAL MODAL V5

套件打包實戰
以 tsdown 重新打包 Vue Final Modal



HUNTER LIU

2026-10-17

Hunter Liu



工作	Colorful Casting
作者	Vue Final Modal
社群	Vue.js Taiwan
演講	MOPCON , Laravel x Vue Conf
網站	hunterliu.tw
聯絡我	@hunterliu1003

PART 1

tsdown

先看打包函式庫的痛點，再跟著一個套件從設定檔走到 npm publish。

1

01 · 為什麼又一個打包工具

打包函式庫的工具，已經不少了

01 · 為什麼又一個打包工具

打包函式庫的工具，已經不少了

VFM 用過的

webpack

Rollup

Vite

01 · 為什麼又一個打包工具

打包函式庫的工具，已經不少了

VFM 用過的

webpack

Rollup

Vite

沒用過的

esbuild

tsup

unbuild

01 · 為什麼又一個打包工具

打包函式庫的工具，已經不少了

VFM 用過的

webpack Rollup Vite

沒用過的

esbuild tsup unbuild

那為什麼還要 tsdown？

01 · 為什麼又一個打包工具

打包函式庫的四個痛點

01 · 為什麼又一個打包工具

打包函式庫的四個痛點

設定繁瑣

Rollup 要湊一堆 plugin，Vite 也要自己疊

01 · 為什麼又一個打包工具

打包函式庫的四個痛點

設定繁瑣

Rollup 要湊一堆 plugin，Vite 也要自己疊

TS 型別宣告複雜

tsc 一次只吐一份，多格式要自己寫腳本

01 · 為什麼又一個打包工具

打包函式庫的四個痛點

設定繁瑣

Rollup 要湊一堆 plugin，Vite 也要自己疊

TS 型別宣告複雜

tsc 一次只吐一份，多格式要自己寫腳本

多格式易出錯

ESM、CJS、UMD 各寫一段，改了會漏

01 · 為什麼又一個打包工具

打包函式庫的四個痛點

設定繁瑣

Rollup 要湊一堆 plugin，Vite 也要自己疊

TS 型別宣告複雜

tsc 一次只吐一份，多格式要自己寫腳本

多格式易出錯

ESM、CJS、UMD 各寫一段，改了會漏

效能瓶頸

JavaScript 寫的打包器，大專案一跑就慢

02 · 認識 TSDOWN

專為現代函式庫而生

THE ELEGANT LIBRARY BUNDLER



tsdown

02 · 認識 TSDOWN

專為現代函式庫而生

THE ELEGANT LIBRARY BUNDLER



極簡設定，開箱即用

tsdown

02 · 認識 TSDOWN

專為現代函式庫而生

THE ELEGANT LIBRARY BUNDLER



- 極簡設定，開箱即用
- 型別宣告自動產生，依賴自動分析

tsdown

02 · 認識 TSDOWN

專為現代函式庫而生

THE ELEGANT LIBRARY BUNDLER



- 極簡設定，開箱即用
- 型別宣告自動產生，依賴自動分析
- 多種格式，一套設定一致輸出

tsdown

02 · 認識 TSDOWN

專為現代函式庫而生

THE ELEGANT LIBRARY BUNDLER



tsdown

- 極簡設定，開箱即用
- 型別宣告自動產生，依賴自動分析
- 多種格式，一套設定一致輸出
- Rust 驅動，底層是 Rolldown

02 · 認識 TSDOWN

專為現代函式庫而生

THE ELEGANT LIBRARY BUNDLER



tsdown

- 極簡設定，開箱即用
- 型別宣告自動產生，依賴自動分析
- 多種格式，一套設定一致輸出
- Rust 驅動，底層是 Rolldown
- 連發布前的 exports、publint、attw 都管

02 · 認識 TSDOWN

它能做什麼

02 · 認識 TSDOWN

它能做什麼



Node.js / CommonJS

```
require('fs')
```

02 · 認識 TSDOWN

它能做什麼



Node.js / CommonJS

`require('fs')`



Vue 元件

`App.vue`

02 · 認識 TSDOWN

它能做什麼



Node.js / CommonJS

`require('fs')`



Vue 元件

`App.vue`



型別宣告

`index.d.ts`

02 · 認識 TSDOWN

它能做什麼

Node.js / CommonJS

`require('fs')`

Vue 元件

`App.vue`

型別宣告

`index.d.ts`

壓縮

`minify · tree shaking`

02 · 認識 TSDOWN

它能做什麼



Node.js / CommonJS

`require('fs')`



Vue 元件

`App.vue`



型別宣告

`index.d.ts`



壓縮

`minify · tree shaking`



打包

`bundle · bundleless`

02 · 認識 TSDOWN

它能做什麼

Node.js / CommonJS

`require('fs')`

Vue 元件

`App.vue`

型別宣告

`index.d.ts`

壓縮

minify · tree shaking

打包

bundle · bundleless

Plugin

Rolldown · Rollup · unplugin

02 · 認識 TSDOWN

它能做什麼

Node.js / CommonJS

`require('fs')`

Vue 元件

`App.vue`

型別宣告

`index.d.ts`

壓縮

`minify · tree shaking`

打包

`bundle · bundleless`

Plugin

`Rolldown · Rollup · unplugin`

還有更多：JSON、WASM、source map、watch…，今天只挑核心的講

02 · 認識 TSDOWN

快速上手

建立新專案

```
pnpm create tsdown@latest
```

裝進既有專案

```
pnpm add -D tsdown
```

一行指令打包

```
pnpm tsdown
# → dist/index.mjs
```

02 · 認識 TSDOWN

快速上手

建立新專案

```
pnpm create tsdown@latest
```

裝進既有專案

```
pnpm add -D tsdown
```

一行指令打包

```
pnpm tsdown
# → dist/index.mjs
```

npm、pnpm、Yarn、Bun 都支援

02 · 認識 TSDOWN

快速上手

建立新專案

```
pnpm create tsdown@latest
```

裝進既有專案

```
pnpm add -D tsdown
```

一行指令打包

```
pnpm tsdown
# → dist/index.mjs
```

- npm、pnpm、Yarn、Bun 都支援
- Node.js 22.18、24.11 或 26 以上，奇數版不支援

02 · 認識 TSDOWN

快速上手

建立新專案

```
pnpm create tsdown@latest
```

裝進既有專案

```
pnpm add -D tsdown
```

一行指令打包

```
pnpm tsdown
# → dist/index.mjs
```

- npm、pnpm、Yarn、Bun 都支援
- Node.js 22.18、24.11 或 26 以上，奇數版不支援
- 沒有設定檔也能跑；有的話，CLI 旗標蓋過去，例如 `tsdown --no-dts`

02 · 認識 TSDOWN

快速上手

建立新專案

```
pnpm create tsdown@latest
```

裝進既有專案

```
pnpm add -D tsdown
```

一行指令打包

```
pnpm tsdown
# → dist/index.mjs
```

- npm、pnpm、Yarn、Bun 都支援
- Node.js 22.18、24.11 或 26 以上，奇數版不支援
- 沒有設定檔也能跑；有的話，CLI 旗標蓋過去，例如 `tsdown --no-dts`
- 常用旗標都有：`--format`、`--watch`、`--minify`

— 03 · TSDOWN 與 ROLLDOWN

同一個核心，兩種用途

— 03 · TSDOWN 與 ROLLDOWN

同一個核心，兩種用途

Rolldown Rust 寫的打包器 · 相容 Rollup 的 API · 對齊 esbuild 的功能

03 · TSDOWN 與 ROLLDOWN

同一個核心，兩種用途

Rolldown Rust 寫的打包器 · 相容 Rollup 的 API · 對齊 esbuild 的功能

應用程式

Vite 8+

dev server、HMR、App build

— 03 · TSDOWN 與 ROLLDOWN

同一個核心，兩種用途

Rolldown Rust 寫的打包器 · 相容 Rollup 的 API · 對齊 esbuild 的功能

應用程式

Vite 8+

dev server、HMR、App build

函式庫

tsdown

型別、格式、exports、發布前檢查

03 · TSDOWN 與 ROLLDOWN

同一個核心，兩種用途

Rolldown Rust 寫的打包器 · 相容 Rollup 的 API · 對齊 esbuild 的功能

應用程式

Vite 8+

dev server、HMR、App build

函式庫

tsdown

型別、格式、exports、發布前檢查

整合

Vite+

`vp build` 用 Vite，`vp pack` 用 tsdown

Rolldown 給能力，tsdown 給預設值

	Rolldown 提供	tsdown 決定

— 03 · TSDOWN 與 ROLLDOWN

Rolldown 給能力，tsdown 給預設值

	Rolldown 提供	tsdown 決定
外部化	`external` 選項	讀 `package.json` 的依賴自動填入

Rolldown 給能力，tsdown 給預設值

	Rolldown 提供	tsdown 決定
外部化	`external` 選項	讀 `package.json` 的依賴自動填入
輸出格式	`format` 選項	一個 format 一次 build，副檔名依 `type` 決定

— 03 · TSDOWN 與 ROLLDOWN

Rolldown 給能力，tsdown 給預設值

	Rolldown 提供	tsdown 決定
外部化	`external` 選項	讀 `package.json` 的依賴自動填入
輸出格式	`format` 選項	一個 format 一次 build，副檔名依 `type` 決定
型別宣告	plugin 介面	用 `rolldown-plugin-dts` 產生 `.d.ts` / `.d.cts`

— 03 · TSDOWN 與 ROLLDOWN

Rolldown 給能力，tsdown 給預設值

	Rolldown 提供	tsdown 決定
外部化	`external` 選項	讀 `package.json` 的依賴自動填入
輸出格式	`format` 選項	一個 format 一次 build，副檔名依 `type` 決定
型別宣告	plugin 介面	用 `rolldown-plugin-dts` 產生 `.d.ts` / `.d.cts`
發布前	不負責	開啟後寫回 `exports`、跑 publint 與 attw

— 03 · TSDOWN 與 ROLLDOWN

Rolldown 給能力，tsdown 給預設值

	Rolldown 提供	tsdown 決定
外部化	`external` 選項	讀 `package.json` 的依賴自動填入
輸出格式	`format` 選項	一個 format 一次 build，副檔名依 `type` 決定
型別宣告	plugin 介面	用 `rolldown-plugin-dts` 產生 `.d.ts` / `.d.cts`
發布前	不負責	開啟後寫回 `exports`、跑 publint 與 attw

> tsdown 負責「產出可以發布的產物」，
> 版本號、changelog、`npm publish` 不在它的範圍。

寫一次 format，產物兩份，副作用一次

format: ['esm', 'cjs'] index.js index.cjs + .d.ts

```ts
// src/config/options.ts
const formats = resolveComma(toArray(format, 'esm'))
return formats.map((fmt, idx) => ({
  ...config,
  copy: idx === 0 ? config.copy : undefined,
  onSuccess: idx === 0 ? config.onSuccess : undefined,
  format: normalizeFormat(fmt),
}))
```



寫一次 format，產物兩份，副作用一次

format: ['esm', 'cjs'] index.js index.cjs + .d.ts

```ts
// src/config/options.ts
const formats = resolveComma(toArray(format, 'esm'))
return formats.map((fmt, idx) => ({
  ...config,
  copy: idx === 0 ? config.copy : undefined,
  onSuccess: idx === 0 ? config.onSuccess : undefined,
  format: normalizeFormat(fmt),
}))
```

- src
  - config 讀設定、補預設值
    - file.ts 找設定檔
    - index.ts resolveConfig 在這
    - **options.ts 預設值、展開 format**
    - workspace.ts 每個 package 一次
  - features 一個功能一支檔案
    - pkg 發布前
      - exports.ts 寫 package.json
      - index.ts 等 format 到齊
      - publint.ts 與 attw 檢查 tarball
    - deps.ts 依賴 external
    - output.ts 副檔名
    - rolldown.ts 組選項與 plugin
    - watch.ts 監看哪些檔
  - build.ts 一次 build 的流程
  - cli.ts 解析參數
  - run.ts bin 入口

寫一次 format，產物兩份，副作用一次

format: ['esm', 'cjs'] index.js index.cjs + .d.ts

```ts
// src/config/options.ts
const formats = resolveComma(toArray(format, 'esm'))
return formats.map((fmt, idx) => ({
  ...config,
  copy: idx === 0 ? config.copy : undefined,
  onSuccess: idx === 0 ? config.onSuccess : undefined,
  format: normalizeFormat(fmt),
}))
```



寫一次 format，產物兩份，副作用一次

format: ['esm', 'cjs'] index.js index.cjs + .d.ts

```ts
// src/build.ts
const bundles = await Promise.all(
  configs.map(o => buildSingle(o, clean, done)),
) // clean 只跑一次
```

- src
  - config 讀設定、補預設值
    - file.ts 找設定檔
    - index.ts resolveConfig 在這
    - options.ts 預設值、展開 format
    - workspace.ts 每個 package 一次
  - features 一個功能一支檔案
    - pkg 發布前
      - exports.ts 寫 package.json
      - index.ts 等 format 到齊
      - publint.ts 與 attw 檢查 tarball
    - deps.ts 依賴 external
    - output.ts 副檔名
    - rolldown.ts 組選項與 plugin
    - watch.ts 監看哪些檔
  - build.ts 一次 build 的流程
  - cli.ts 解析參數
  - run.ts bin 入口

寫一次 format，產物兩份，副作用一次

`format: ['esm', 'cjs']` index.js index.cjs + .d.ts

```ts
// src/build.ts
const bundles = await Promise.all(
  configs.map(o => buildSingle(o, clean, done)),
) // clean 只跑一次
```

副檔名也是它決定：platform 是 node 一律 .mjs / .cjs；VFM 是 neutral 加 "type": "module"，所以是 .js / .cjs



要不要產型別，它看你的 package.json

三個訊號有一個就自動產 `types` `exports.types` `declaration`

```ts
// src/config/options.ts · 設定裡沒寫 dts 的時候
if (dts == null) {
  if (pkg?.types || pkg?.typings || hasExportsTypes(pkg?.exports))
    dts = true
  } else if (tsconfig) {
    const { config } = readTsconfig(tsconfig)
    dts = !!config.compilerOptions?.declaration
  }
}
```



要不要產型別，它看你的 package.json

三個訊號有一個就自動產 `types` `exports.types` `declaration`

```ts
// src/config/options.ts · 設定裡沒寫 dts 的時候
if (dts == null) {
  if (pkg?.types || pkg?.typings || hasExportsTypes(pkg?.exports))
    dts = true
  } else if (tsconfig) {
    const { config } = readTsconfig(tsconfig)
    dts = !!config.compilerOptions?.declaration
  }
}
```



要不要產型別，它看你的 package.json

三個訊號有一個就自動產 `types` `exports.types` `declaration`

```ts
// src/config/options.ts · 設定裡沒寫 dts 的時候
if (dts == null) {
  if (pkg?.types || pkg?.typings || hasExportsTypes(pkg?.exports))
    dts = true
  } else if (tsconfig) {
    const { config } = readTsconfig(tsconfig)
    dts = !!config.compilerOptions?.declaration
  }
}
```



要不要產型別，它看你的 package.json

三個訊號有一個就自動產 `types` `exports.types` `declaration`

```ts
// src/config/options.ts · 設定裡沒寫 dts 的時候
if (dts == null) {
  if (pkg?.types || pkg?.typings || hasExportsTypes(pkg?.exports))
    dts = true
  } else if (tsconfig) {
    const { config } = readTsconfig(tsconfig)
    dts = !!config.compilerOptions?.declaration
  }
}
```

Vue 元件庫多一行

`dts: { vue: true }`，型別交給 vue-tsc，SFC 的 props 才推得出來；ESM 配 `.d.ts`、CJS 配 `.d.cts`



依賴自動外部化，打進去的會寫給你看

VFM 的依賴 `vue` `focus-trap` `vue-use-template` → 只剩 import

```ts
// src/features/deps.ts · 這個 import 要不要外部化
if (deps.includes(id) || deps.some(dep => id.startsWith(`${dep}/`)
  return true // dependencies、peerDependencies，連子路徑都算
if (importer && RE_DTS.test(importer) && !id.startsWith('@types/')
  const typesName = getTypesPackageName(id)
  if (typesName && deps.includes(typesName)) return true
} // d.ts 裡 import 的套件，有對應的 @types 也外部化
```



依賴自動外部化，打進去的會寫給你看

VFM 的依賴 `vue` `focus-trap` `vue-use-template` → 只剩 import

```ts
// src/features/deps.ts · 這個 import 要不要外部化
if (deps.includes(id) || deps.some(dep => id.startsWith(`${dep}/`)
  return true // dependencies、peerDependencies，連子路徑都算
if (importer && RE_DTS.test(importer) && !id.startsWith('@types/')
  const typesName = getTypesPackageName(id)
  if (typesName && deps.includes(typesName)) return true
} // d.ts 裡 import 的套件，有對應的 @types 也外部化
```



依賴自動外部化，打進去的會寫給你看

VFM 的依賴 `vue` `focus-trap` `vue-use-template` → 只剩 import

```ts
// src/features/deps.ts · 這個 import 要不要外部化
if (deps.includes(id) || deps.some(dep => id.startsWith(`${dep}/`)))
  return true // dependencies、peerDependencies，連子路徑都算
if (importer && RE_DTS.test(importer) && !id.startsWith('@types/')) {
  const typesName = getTypesPackageName(id)
  if (typesName && deps.includes(typesName)) return true
} // d.ts 裡 import 的套件，有對應的 @types 也外部化
```



依賴自動外部化，打進去的會寫給你看

VFM 的依賴 `vue` `focus-trap` `vue-use-template` → 只剩 import

```ts
// src/features/deps.ts · 這個 import 要不要外部化
if (deps.includes(id) || deps.some(dep => id.startsWith(`${dep}/`)))
  return true // dependencies、peerDependencies，連子路徑都算
if (importer && RE_DTS.test(importer) && !id.startsWith('@types/')) {
  const typesName = getTypesPackageName(id)
  if (typesName && deps.includes(typesName)) return true
} // d.ts 裡 import 的套件，有對應的 @types 也外部化
```

VFM 的 UMD 是例外

`deps.alwaysBundle` 把 vue-use-template 與 scroll-lock 打進 UMD，CDN 使用者沒有 build step；有開 `exports` 的套件，打進去的套件與版本會寫進 `inlinedDependencies`



CSS 零設定，而且排在你的 plugin 之後

Vue 一個 plugin，CSS 一行都不用寫

SFC 交給 unplugin-vue/rolldown；偵測到 @tsdown/css 就掛上，SFC 的 style 抽成 style.css，有開 exports 還會補一條 ./style.css

```
// src/features/rolldown.ts · plugin 順序
plugins.push(DepsPlugin(config, bundle)) // external
if (dts) plugins.push(dtsPlugin(options)) // 型別
if (hasCss) plugins.push(...cssPlugins.pre) // @tsdown/css
plugins.push(ReportPlugin(config))
plugins.push(userPlugins) // 你的 unplugin-vue
plugins.push(...cssPostPlugins) // 收 CSS，排最後
```



CSS 零設定，而且排在你的 plugin 之後

Vue 一個 plugin，CSS 一行都不用寫

SFC 交給 unplugin-vue/rolldown；偵測到 @tsdown/css 就掛上，SFC 的 style 抽成 style.css，有開 exports 還會補一條 ./style.css

```ts
// src/features/rolldown.ts · plugin 順序
plugins.push(DepsPlugin(config, bundle)) // external
if (dts) plugins.push(dtsPlugin(options)) // 型別
if (hasCss) plugins.push(...cssPlugins.pre) // @tsdown/css
plugins.push(ReportPlugin(config))
plugins.push(userPlugins) // 你的 unplugin-vue
plugins.push(...cssPostPlugins) // 收 CSS，排最後
```



CSS 零設定，而且排在你的 plugin 之後

Vue 一個 plugin，CSS 一行都不用寫

SFC 交給 unplugin-vue/rolldown；偵測到 @tsdown/css 就掛上，SFC 的 style 抽成 style.css，有開 exports 還會補一條 ./style.css

```
// src/features/rolldown.ts · plugin 順序
plugins.push(DepsPlugin(config, bundle)) // external
if (dts) plugins.push(dtsPlugin(options)) // 型別
if (hasCss) plugins.push(...cssPlugins.pre) // @tsdown/css
plugins.push(ReportPlugin(config))
plugins.push(userPlugins) // 你的 unplugin-vue
plugins.push(...cssPostPlugins) // 收 CSS，排最後
```



CSS 零設定，而且排在你的 plugin 之後

Vue 一個 plugin，CSS 一行都不用寫

SFC 交給 unplugin-vue/rolldown；偵測到 @tsdown/css 就掛上，SFC 的 style 抽成 style.css，有開 exports 還會補一條 ./style.css

```
// src/features/rolldown.ts · plugin 順序
plugins.push(DepsPlugin(config, bundle)) // external
if (dts) plugins.push(dtsPlugin(options)) // 型別
if (hasCss) plugins.push(...cssPlugins.pre) // @tsdown/css
plugins.push(ReportPlugin(config))
plugins.push(userPlugins) // 你的 unplugin-vue
plugins.push(...cssPostPlugins) // 收 CSS，排最後
```



一行 exports，package.json 它來寫

```ts
// tsdown.config.ts
export default defineConfig({
  exports: true,
  format: ['cjs', 'esm'],
})
```

```jsonc
// package.json · 這些欄位由 tsdown 寫回
{
  "main": "./dist/index.cjs",
  "module": "./dist/index.mjs",
  "types": "./dist/index.d.cts",
  "exports": {
    ".": { "import": "./dist/index.mjs", "require": "./dist/index.cjs" },
    "./package.json": "./package.json"
  }
}
```

一行 exports，package.json 它來寫

```ts
// tsdown.config.ts · 多一個入口
export default defineConfig({
  exports: true,
  format: ['cjs', 'esm'],
  entry: ['src/index.ts',
    'src/foo.ts'],
})
```

```jsonc
// package.json · 自動多一個子路徑
{
  "main": "./dist/index.cjs",
  "module": "./dist/index.mjs",
  "types": "./dist/index.d.cts",
  "exports": {
    ".": { "import": "./dist/index.mjs", "require": "./dist/index.cjs" },
    "./foo": { "import": "./dist/foo.mjs", "require": "./dist/foo.cjs" },
    "./package.json": "./package.json"
  }
}
```

一行 exports，package.json 它來寫

```ts
// tsdown.config.ts · 開發時指向原始碼
export default defineConfig({
  exports: { devExports: true },
  format: ['cjs', 'esm'],
  entry: ['src/index.ts',
    'src/foo.ts'],
})
```

```jsonc
// package.json · 開發指 src，發布時由 publishConfig 換回 dist
{
  "main": "./dist/index.cjs",
  "module": "./dist/index.mjs",
  "types": "./dist/index.d.cts",
  "exports": {
    ".": "./src/index.ts",
    "./foo": "./src/foo.ts",
    "./package.json": "./package.json"
  },
  "publishConfig": {
    "exports": {
      ".": { "import": "./dist/index.mjs", "require": "./dist/index.cjs" },
      "./foo": { "import": "./dist/foo.mjs", "require": "./dist/foo.cjs" },
      "./package.json": "./package.json"
    }
  }
}
```

一行 exports，package.json 它來寫

```ts
// tsdown.config.ts · 開發時指向原始碼
export default defineConfig({
  exports: { devExports: true },
  format: ['cjs', 'esm'],
  entry: ['src/index.ts',
    'src/foo.ts'],
})
```

一個坑

純 ESM 時 main、module 會拿掉，舊的 types 卻留著指向不存在的 .d.cts；開 legacy: true 或自己刪

```jsonc
// package.json · 開發指 src，發布時由 publishConfig 換回 dist
{
  "main": "./dist/index.cjs",
  "module": "./dist/index.mjs",
  "types": "./dist/index.d.cts",
  "exports": {
    ".": "./src/index.ts",
    "./foo": "./src/foo.ts",
    "./package.json": "./package.json"
  },
  "publishConfig": {
    "exports": {
      ".": { "import": "./dist/index.mjs", "require": "./dist/index.cjs" },
      "./foo": { "import": "./dist/foo.mjs", "require": "./dist/foo.cjs" },
      "./package.json": "./package.json"
    }
  }
}
```

兩個檢查，對著真正的 tarball 跑

兩個檢查，對著真正的 tarball 跑

publint · publint.dev

對 npm pack 出來的那一包檢查 package.json：main、types、exports 指到的檔案在不在、格式有沒有擺錯

兩個檢查，對著真正的 tarball 跑

publint · publint.dev

對 npm pack 出來的那一包檢查 package.json：main、types、exports 指到的檔案在不在、格式有沒有擺錯

attw · Are the types wrong?

模擬 node10、node16 的 import 與 require、bundler 四種解析去找型別，抓使用者裝了才會炸的錯

兩個檢查，對著真正的 tarball 跑

`publint · publint.dev`

對 npm pack 出來的那一包檢查 package.json：main、types、exports 指到的檔案在不在、格式有沒有擺錯

`attw · Are the types wrong?`

模擬 node10、node16 的 import 與 require、bundler 四種解析去找型別，抓使用者裝了才會炸的錯

scroll-lock 實跑：純 ESM，舊的 types 沒刪

```
$ publint
Errors:
1. pkg.types is ./dist/index.d.cts
   but the file does not exist.

$ attw --pack
⚠ A require call resolved to
   an ESM JavaScript file
node16 (from CJS)  ⚠ ESM (dynamic import only)
```

設 publint: 'ci-only'、attw: 'ci-only'，只在 CI=true 時跑，本機 build 不會被拖慢

兩個檢查，對著真正的 tarball 跑

`publint · publint.dev`

對 npm pack 出來的那一包檢查 package.json：main、types、exports 指到的檔案在不在、格式有沒有擺錯

`attw · Are the types wrong?`

模擬 node10、node16 的 import 與 require、bundler 四種解析去找型別，抓使用者裝了才會炸的錯

scroll-lock 實跑：純 ESM，舊的 types 沒刪

```
$ publint
Linting... (沒有 Errors)

$ attw --pack
No problems found 🌟
node16 (from CJS)  🟢  (CJS)
```

設 publint: 'ci-only'、attw: 'ci-only'，只在 CI=true 時跑，本機 build 不會被拖慢

兩個檢查，對著真正的 tarball 跑

`publint · publint.dev`

對 npm pack 出來的那一包檢查 package.json：main、types、exports 指到的檔案在不在、格式有沒有擺錯

`attw · Are the types wrong?`

模擬 node10、node16 的 import 與 require、bundler 四種解析去找型別，抓使用者裝了才會炸的錯

什麼時候跑：所有 format 到齊 → 寫 exports → npm pack → 兩個檢查平行跑同一包；檢查的是真正會上傳的內容，不是 dist

scroll-lock 實跑：純 ESM，舊的 types 沒刪

```
$ publint
Linting... (沒有 Errors)

$ attw --pack
No problems found 🌟
node16 (from CJS)  🟢  (CJS)
```

設 publint: 'ci-only'、attw: 'ci-only'，只在 CI=true 時跑，本機 build 不會被拖慢

05 · 現況與證據

現在有多少人在用

05 · 現況與證據

現在有多少人在用

NPM 月下載

2,014 萬

2026-09-13 到 10-12 · api.npmjs.org

GITHUB STARS

4,300

rolldown/tsdown · 2026-10-15

十五個月的成長

82×

月下載 24.5 萬 → 2,014 萬

05 · 現況與證據

現在有多少人在用

NPM 月下載

2,014 萬

2026-09-13 到 10-12 · api.npmjs.org

GITHUB STARS

4,300

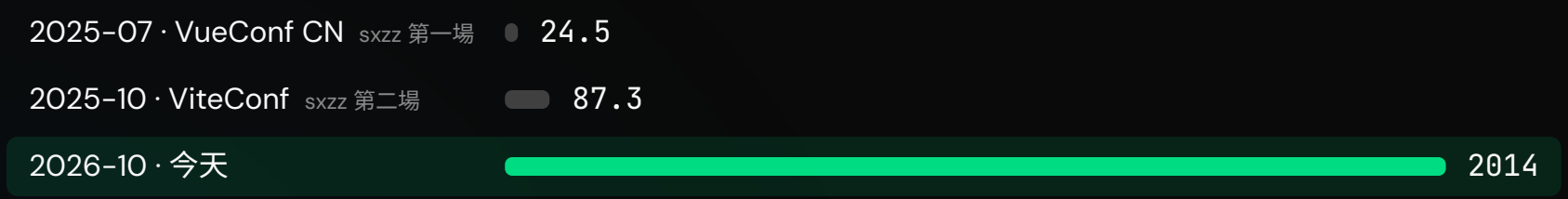
rolldown/tsdown · 2026-10-15

十五個月的成長

82×

月下載 24.5 萬 → 2,014 萬

NPM 月下載，單位：萬



— 05 · 現況與證據

現在有多少人在用

NPM 月下載

2,014 萬

2026-09-13 到 10-12 · api.npmjs.org

GITHUB STARS

4,300

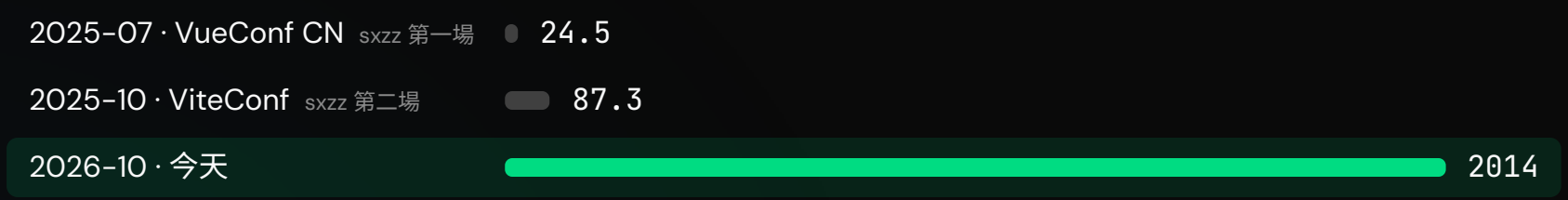
rolldown/tsdown · 2026-10-15

十五個月的成長

82×

月下載 24.5 萬 → 2,014 萬

NPM 月下載，單位：萬



背後是整個 Vite 生態：

- Vite 8 內建 Rolldown · 2026-03
- Vite+ 的 vp pack 以 tsdown 為核心

PART 2

Vue Final Modal v5

換打包工具，也換核心，API 不變。



The most powerful modal library for Vue

Vue Final Modal 是什麼

The most powerful modal library for Vue

Vue Final Modal 是什麼

上 NPM 至今

6 年

2020-06 起 · Vue 2 → 3 · JS → TS

NPM 月下載

28.6 萬

09-13 到 10-12 · api.npmjs.org

GITHUB STARS

1,010

GitHub · 2026-10-15



The most powerful modal library for Vue

Vue Final Modal 是什麼

上 NPM 至今

6 年

2020-06 起 · Vue 2 → 3 · JS → TS

NPM 月下載

28.6 萬

09-13 到 10-12 · api.npmjs.org

GITHUB STARS

1,010

GitHub · 2026-10-15

核心概念

盡量不帶預設的 CSS 樣式

切版的彈性留給前端工程師

The most powerful modal library for Vue

Vue Final Modal 是什麼

上 NPM 至今

6 年

2020-06 起 · Vue 2 → 3 · JS → TS

NPM 月下載

28.6 萬

09-13 到 10-12 · api.npmjs.org

GITHUB STARS

1,010

GitHub · 2026-10-15

核心概念

盡量不帶預設的 CSS 樣式

切版的彈性留給前端工程師

背後做掉的事

- 點擊外部關閉
- Esc 關閉
- 無障礙與易用性
- 鎖住頁面捲動
- 巢狀 z-index 自動計算
- 開關過渡動畫
- 等等

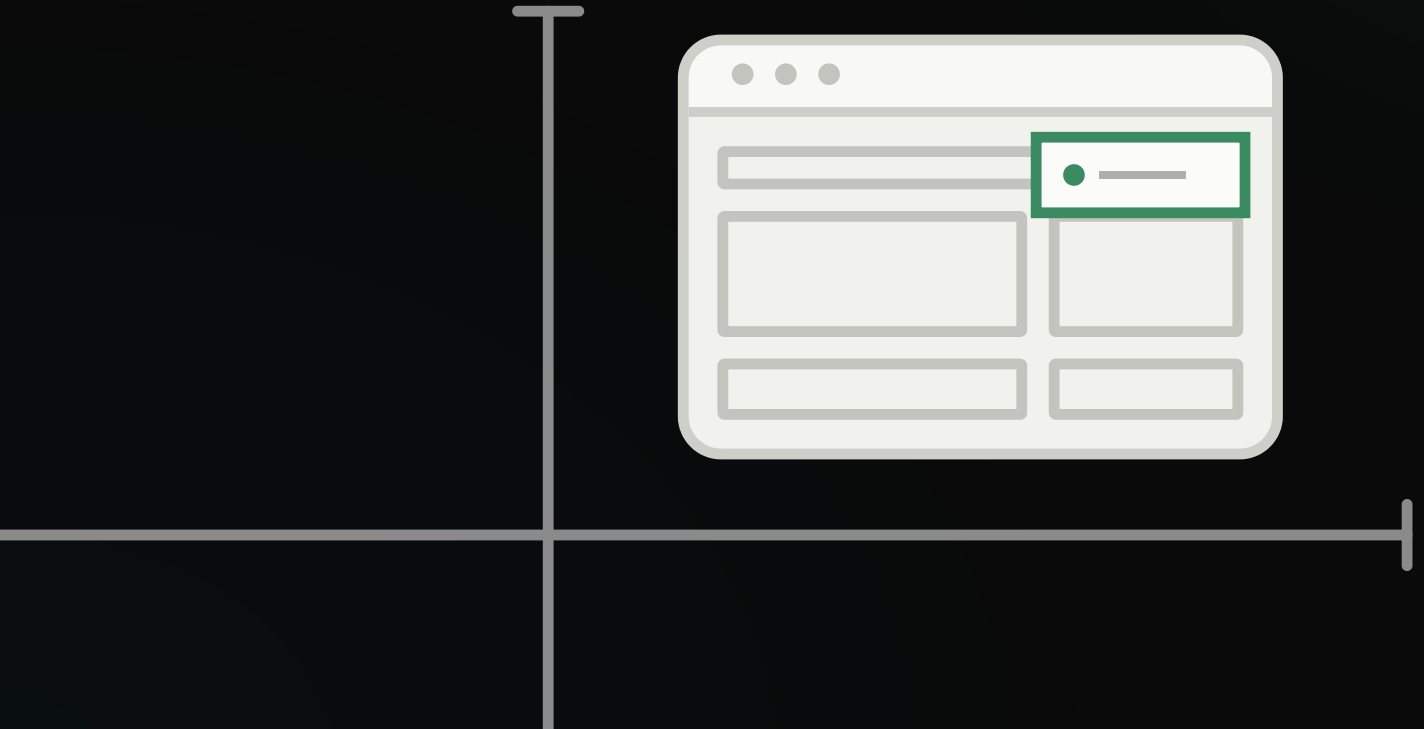
UX · 象限 · MODAL 廣義的分類



改繪自 NN/g · The Anatomy of Popups · nngroup.com

Non-overlay

（沒有遮罩）



Nonmodal

（能操作背景）

Non-overlay

（沒有遮罩）



Nonmodal

（能操作背景）

Non-overlay

（沒有遮罩）

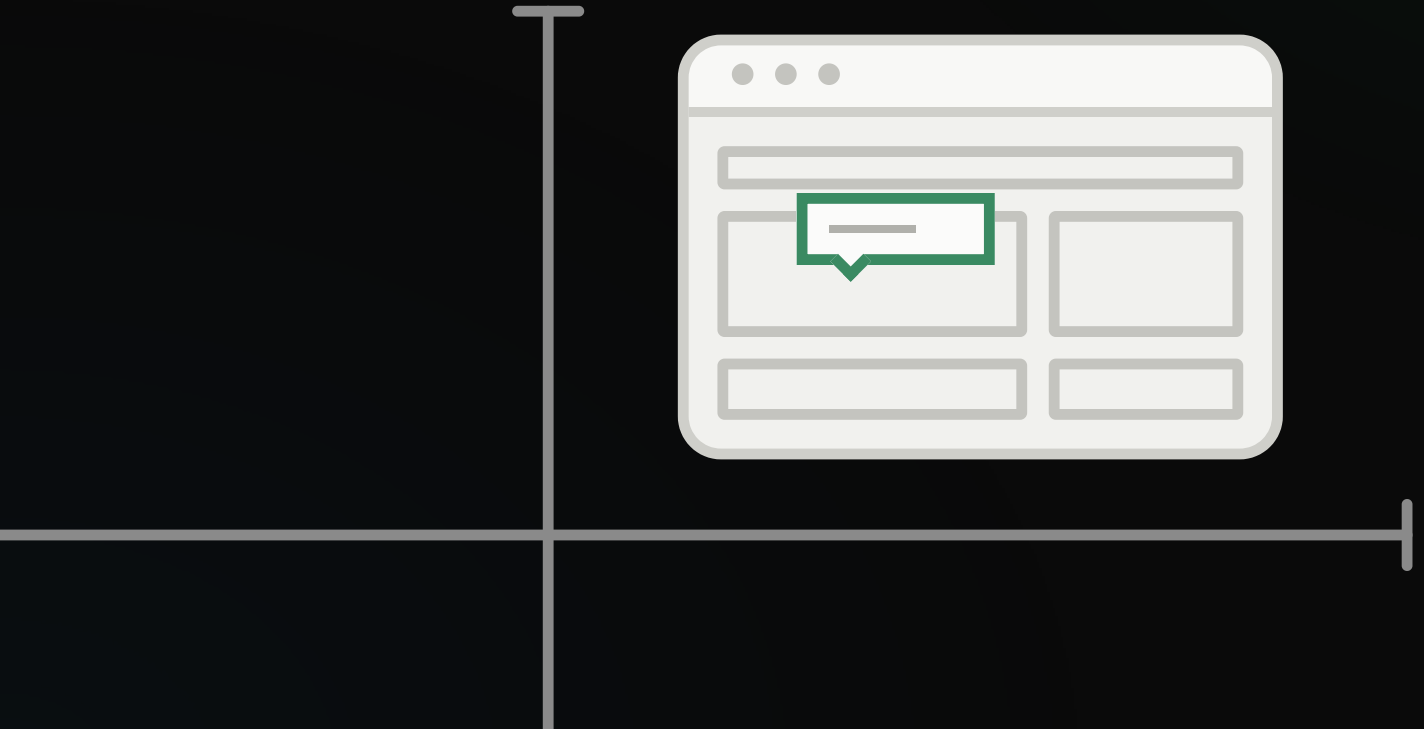


Nonmodal

（能操作背景）

Non-overlay

（沒有遮罩）

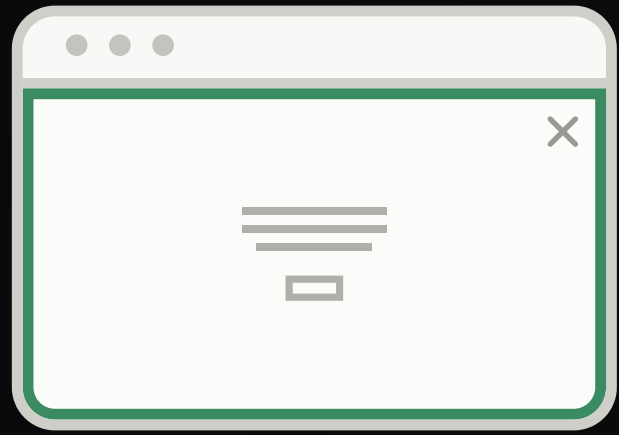


Nonmodal

（能操作背景）

Non-overlay

（沒有遮罩）



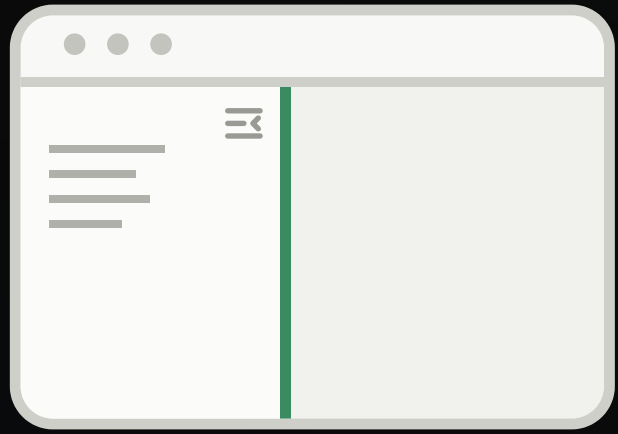


Modal

（不能操作背景）

Non-overlay

（沒有遮罩）





Modal

（不能操作背景）

Non-overlay

（沒有遮罩）

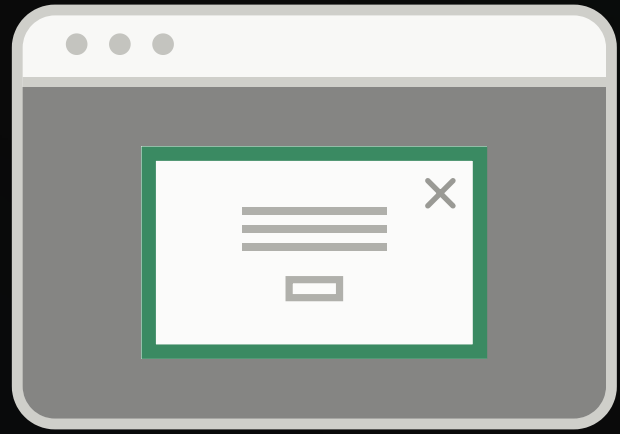




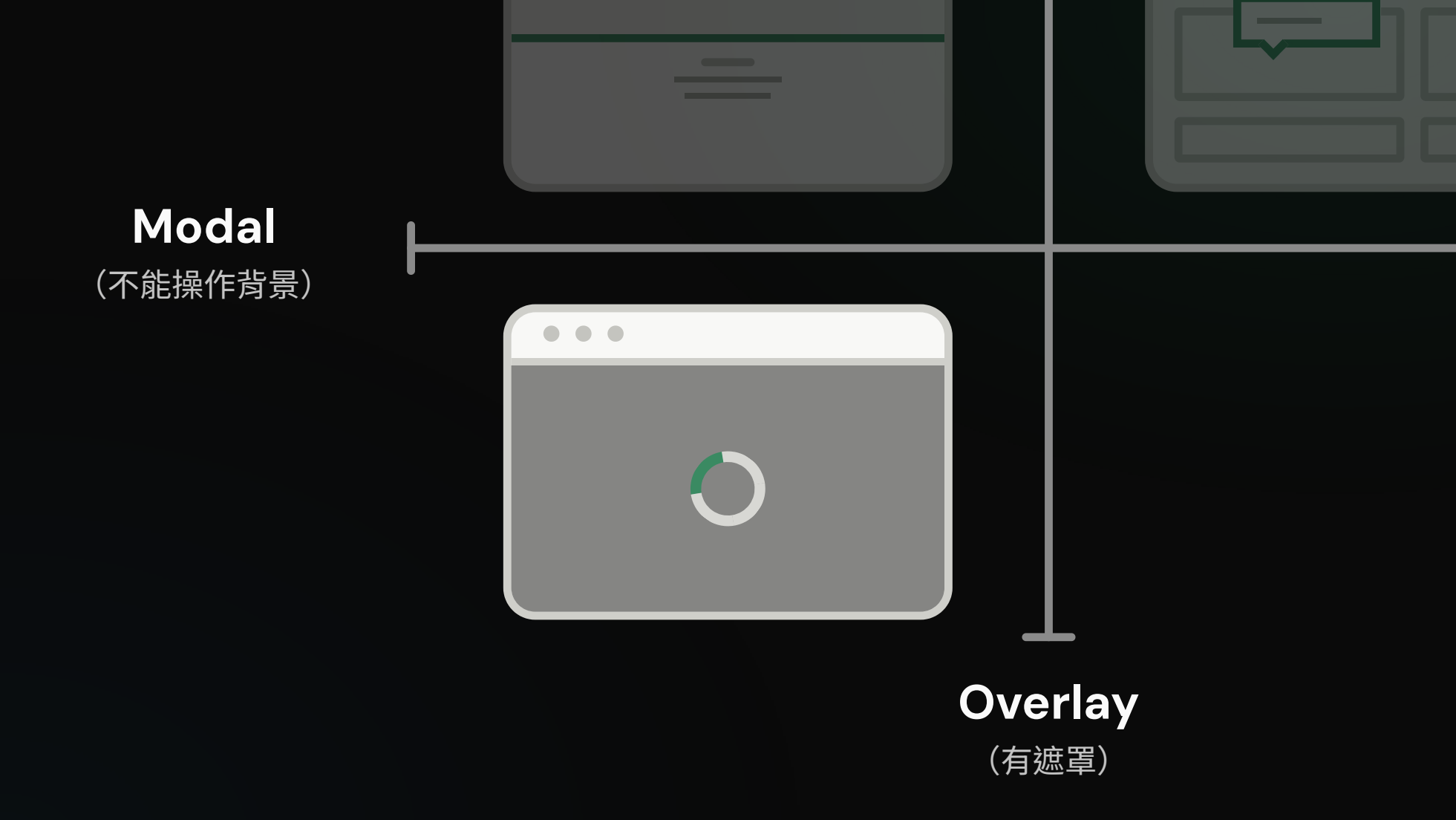
Modal

（不能操作背景）

Modal
（不能操作背景）



Overlay
（有遮罩）


Modal
（不能操作背景）
Overlay
（有遮罩）

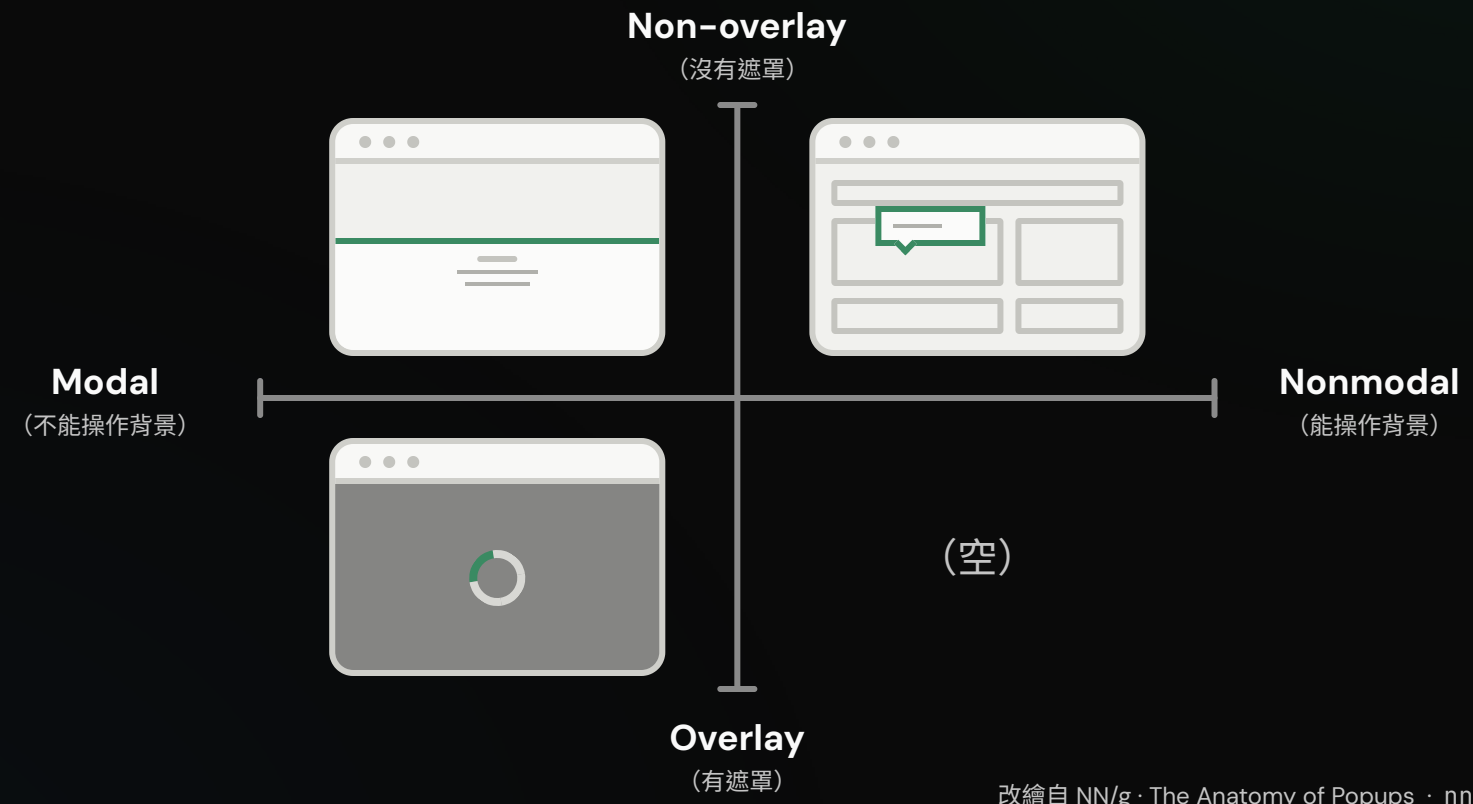


Nonmodal
（能操作背景）

（空）

Overlay
（有遮罩）

UX · 象限 · MODAL 廣義的分類



改繪自 NN/g · The Anatomy of Popups · nngroup.com

UX · 決策樹 · 選對類型

用決策樹選對 Modal 的類型

七個問題，每個出口都是最適合那個情境的元件

UX · 決策樹 · 選對類型

用決策樹選對 Modal 的類型



Q1 · 任一成立就是 YES

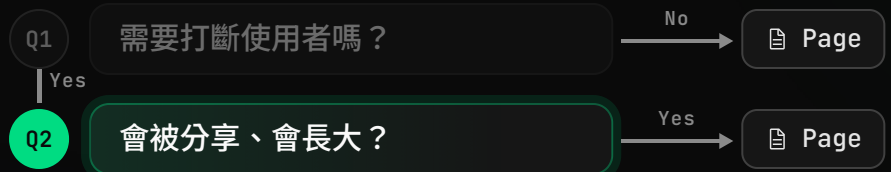
需要打斷使用者嗎？

- 需要抓住使用者的注意力
- 使用者只從少數幾個選項中挑一個
- 要提供額外內容，又不能離開主頁面
- 繼續之前，要先收集少量資訊
- 內容緊急

No → Page

UX · 決策樹 · 選對類型

用決策樹選對 Modal 的類型



Q2 · 任一成立就是 YES

會被分享、會長大？

- 使用者想把內容加入書籤或分享
- 使用者離開流程後，可能再回來
- 有複雜的操作互相影響
- 未來會擴充其他操作、任務或內容

Yes → Page

UX · 決策樹 · 選對類型

用決策樹選對 Modal 的類型



Q3 · 任一成立就是 YES

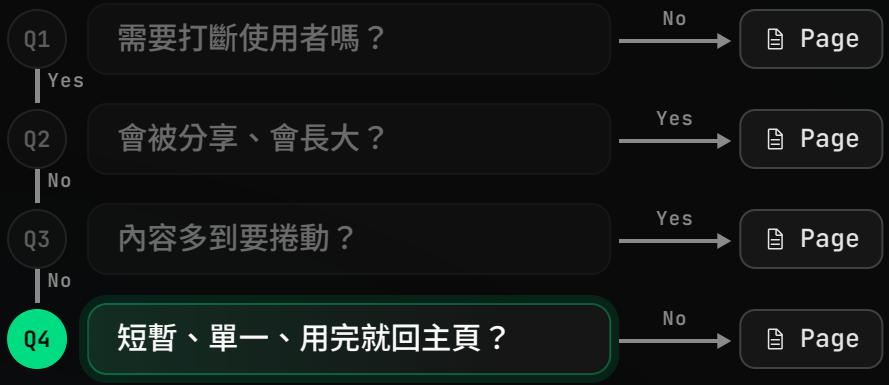
內容多到要捲動？

- 畫面中會塞入大量元素
- 內容高度超過裝置高度
- 內容需要捲動
- 有兩個以上的操作要使用者完成
- 要使用者完成一項以上的任務

Yes → Page

UX · 決策樹 · 選對類型

用決策樹選對 Modal 的類型



Q4 · 任一成立就是 YES

短暫、單一、用完就回主頁？

- 使用者常在功能之間來回切換
- 使用者與功能的互動很單一
- 內容是暫時的
- 之後注意力要回到主頁面
- 內容簡短
- 內容位在 sitemap 的最深層
- 有其他充分的理由

No → Page

UX · 決策樹 · 選對類型

用決策樹選對 Modal 的類型



Q5 · 任一成立就是 YES

非擋住流程不可？

- 使用者需要「立刻」注意到
- 轉移注意力，任務反而更容易完成
- 這個任務是接下來流程的必要條件
- 先完成它，能明顯省下後面的操作
- 任務有明確步驟，有起點也有終點

No → Nonmodal

UX · 決策樹 · 選對類型

用決策樹選對 Modal 的類型



Q6 · 任一成立就是 YES

只是告知結果？

- 成功訊息：「已儲存」
- 按鈕直接變成「已儲存」

Yes → Toast / Inline

UX · 決策樹 · 選對類型

用決策樹選對 Modal 的類型



Q7 · 任一成立就是 YES

要收集資訊或確認操作？

- 收集使用者資訊
- 確認使用者的操作
- 嚴重或不可逆的操作
- 填寫表單或做決定
- 防止嚴重錯誤的警告

Yes → Dialog

No → Bottom Sheet · Nav Drawer

v5 改了什麼



打包換成 tsdown

核心：vue-use-template

行為對齊原生 dialog

新的 scroll-lock

正式支援 SSR

Nuxt module 併入主套件

API 向下相容

更多 e2e 測試

06 · 打包換成 TSDOWN

三代打包工具，三份設定檔

2020-06 · V0

Vue 2 · Vue CLI

拿 App 工具打包函式庫，只產 UMD

```
// package.json · Vue CLI
"scripts": {
  "build:lib": "vue-cli-service build --target lib \\
    --name VueFinalModal ./plugin.js --dest lib",
},
"main": "./lib/VueFinalModal.umd.min.js",
"unpkg": "./lib/VueFinalModal.umd.min.js",
```

2020-08 → 2021-03 · V0.10 到 V3

Vue 2 與 Vue 3 兩條線 · Rollup

手接 7 個 plugin，型別手寫 d.ts

2023-02 · V4

Vue 3 · Vite library mode

改寫成 TypeScript，型別自動產生

06 · 打包換成 TSDOWN

三代打包工具，三份設定檔

2020-06 · V0

Vue 2 · Vue CLI

拿 App 工具打包函式庫，只產 UMD

2020-08 → 2021-03 · V0.10 到 V3

Vue 2 與 Vue 3 兩條線 · Rollup

手接 7 個 plugin，型別手寫 d.ts

2023-02 · V4

Vue 3 · Vite library mode

改寫成 TypeScript，型別自動產生

```
// rollup.config.js · 46 行
const plugins = [
  NodeResolve(), VuePlugin(), cleanup(), terser(), PostCSS(), commonjs(), sizes(),
  babel({ babelHelpers: 'bundled', extensions: ['.js', '.vue'] }),
]
export default {
  input: 'lib/index.js',
  output: [
    { file: pkg.main, format: 'umd', name: 'VueFinalModal',
      globals: { vue: 'Vue' } },
    { file: pkg.module, format: 'esm' },
  ],
  plugins, external: ['vue'],
}
```

06 · 打包換成 TSDOWN

三代打包工具，三份設定檔

2020-06 · V0

Vue 2 · Vue CLI

拿 App 工具打包函式庫，只產 UMD

2020-08 → 2021-03 · V0.10 到 V3

Vue 2 與 Vue 3 兩條線 · Rollup

手接 7 個 plugin，型別手寫 d.ts

2023-02 · V4

Vue 3 · Vite library mode

改寫成 TypeScript，型別自動產生

```
// vite.config.ts · 56 行
export default defineConfig({
  plugins: [VueMacros({ plugins: { vue: Vue() } }),
    DefineOptions(), dts({ include: 'src' })],
  build: {
    lib: { entry: 'src/index.ts', name, fileName: f => `index.${f}.js` },
    rollupOptions: {
      external: ['vue', '@vueuse/core', 'focus-trap' /* … */],
      output: { globals: { 'vue': 'Vue', '@vueuse/core': 'VueUse' /* … */ } },
    },
  },
})
```

06 · 打包換成 TSDOWN

三代打包工具，三份設定檔

2020-06 · V0

Vue 2 · Vue CLI

拿 App 工具打包函式庫，只產 UMD

2020-08 → 2021-03 · V0.10 到 V3

Vue 2 與 Vue 3 兩條線 · Rollup

手接 7 個 plugin，型別手寫 d.ts

2023-02 · V4

Vue 3 · Vite library mode

改寫成 TypeScript，型別自動產生

```
// vite.config.ts · 56 行
export default defineConfig({
  plugins: [VueMacros({ plugins: { vue: Vue() } }),
    DefineOptions(), dts({ include: 'src' })],
  build: {
    lib: { entry: 'src/index.ts', name, fileName: f => `index.${f}.js` },
    rollupOptions: {
      external: ['vue', '@vueuse/core', 'focus-trap' /* … */],
      output: { globals: { 'vue': 'Vue', '@vueuse/core': 'VueUse' /* … */ } },
    },
  },
})
```

> **設定越堆越多，打包工具本身變成技術債。**

06 · 打包換成 TSDOWN

v5 的 tsdown 設定

dist/

index.js ESM

index.cjs CJS

index.d.ts ESM 型別

index.d.cts CJS 型別

style.css @tsdown/css

index.umd.js CDN

nuxt/ Nuxt module

```ts
// tsdown.config.ts
export default defineConfig({
  entry: ['./src/index.ts'],
  platform: 'neutral',
  format: ['esm', 'cjs'],
  plugins: [Vue()],
  dts: { vue: true },
})
```

06 · 打包換成 TSDOWN

v5 的 tsdown 設定

dist/

index.js ESM

index.cjs CJS

index.d.ts ESM 型別

index.d.cts CJS 型別

style.css @tsdown/css

index.umd.js CDN

nuxt/ Nuxt module

```
// package.json · pnpm add -D @tsdown/css，設定檔一行不用改
"devDependencies": {
  "@tsdown/css": "^0.22.5"
},
// build 完 tsdown 自動補的
"exports": {
  "./style.css": "./dist/style.css"
}
```

06 · 打包換成 TSDOWN

v5 的 tsdown 設定

dist/

index.js ESM

index.cjs CJS

index.d.ts ESM 型別

index.d.cts CJS 型別

style.css @tsdown/css

index.umd.js CDN

nuxt/ Nuxt module

```
// tsdown.config.ts
{
  ...library(),
  format: ['umd'],
  deps: { alwaysBundle: ['vue-use-template', '@hunterliu/scroll-lock'] },
  minify: true,
  outputOptions: {
    name: 'VueFinalModal',
    globals: { 'vue': 'Vue', '@vueuse/core': 'VueUse' /* … */ },
  },
}
```

06 · 打包換成 TSDOWN

v5 的 tsdown 設定

dist/

index.js ESM

index.cjs CJS

index.d.ts ESM 型別

index.d.cts CJS 型別

style.css @tsdown/css

index.umd.js CDN

nuxt/ Nuxt module

```ts
// tsdown.config.ts
{
  entry: {
    'nuxt/module': './src/nuxt/module.ts',
    'nuxt/runtime/plugin': './src/nuxt/runtime/plugin.ts',
    'nuxt/runtime/server': './src/nuxt/runtime/server.ts',
  },
  platform: 'node',
  format: ['esm'],
  deps: { neverBundle: [/^@nuxt\//, /^nuxt(\/|$)/, /^#/, 'vue-final-modal'] },
  dts: true,
  clean: false,
}
```

06 · 打包換成 TSDOWN

一個 monorepo，一次 build

根目錄的 tsdown.config.ts

```ts
import { defineConfig } from 'tsdown'

export default defineConfig({
  workspace: 'packages/*',
})
```

- packages/
  - scroll-lock/ tsdown.config.ts
  - vue-use-template/ tsdown.config.ts
  - vue-final-modal/ tsdown.config.ts

06 · 打包換成 TSDOWN

一個 monorepo，一次 build

根目錄的 tsdown.config.ts

```ts
import { defineConfig } from 'tsdown'

export default defineConfig({
  workspace: 'packages/*',
})
```

workspace 做的事

掃 packages/*，每個套件用自己的設定、各自 build，平行跑

scroll-lock 幾行就夠，VFM 三組 build，根目錄不用知道

- packages/
  - scroll-lock/ tsdown.config.ts
  - vue-use-template/ tsdown.config.ts
  - vue-final-modal/ tsdown.config.ts

06 · 打包換成 TSDOWN

一個 monorepo，一次 build

根目錄的 tsdown.config.ts

```ts
import { defineConfig } from 'tsdown'

export default defineConfig({
  workspace: 'packages/*',
})
```

```
packages/
  scroll-lock/ tsdown.config.ts
  vue-use-template/ tsdown.config.ts
  vue-final-modal/ tsdown.config.ts
```

workspace 做的事

掃 packages/*，每個套件用自己的設定、各自 build，平行跑

scroll-lock 幾行就夠，VFM 三組 build，根目錄不用知道

平行沒有順序，所以分兩次

```
tsdown -F vue-use-template -F @hunterliu/scroll-lock
tsdown -F vue-final-modal
```

UMD 要把另外兩包的 dist 打進去，它們得先好

— 07 · V5 概覽

一個入口，兩種 modal

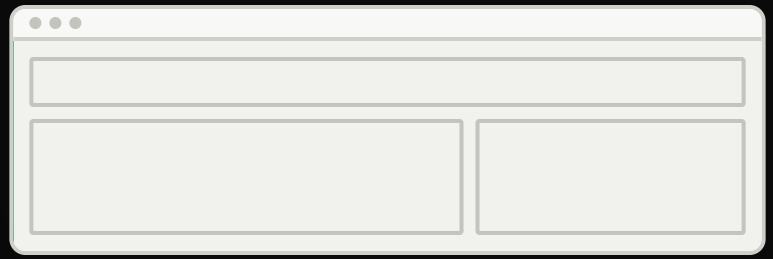
```
const { open } = useModal({
  component: …,
})
open()
```

component 可以是

- VueFinalModal
- VueFinalDialog
- 用 VueFinalModal 做的 HOC
  - Dialog
  - NavDrawer
  - BottomSheet
  - Spinner
- 你自己的 Modal 元件

push →

```
<ModalsContainer />          modals: 0
```



— 07 · V5 概覽

一個入口，兩種 modal

```
const { open } = useModal({
  component: VueFinalModal,
})
open()
```

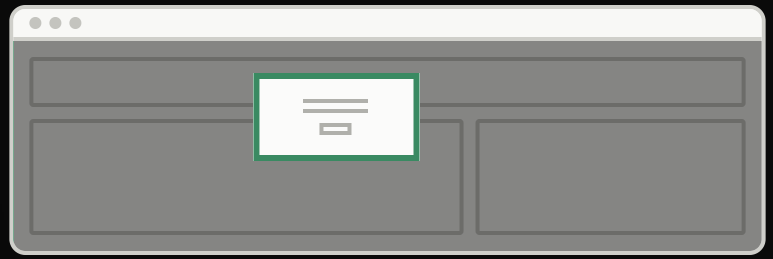
component 可以是

- VueFinalModal — opened
- VueFinalDialog
- 用 VueFinalModal 做的 HOC
  - Dialog
  - NavDrawer
  - BottomSheet
  - Spinner
- 你自己的 Modal 元件

push →

<ModalsContainer /> modals: 1

1 VueFinalModal



— 07 · V5 概覽

一個入口，兩種 modal

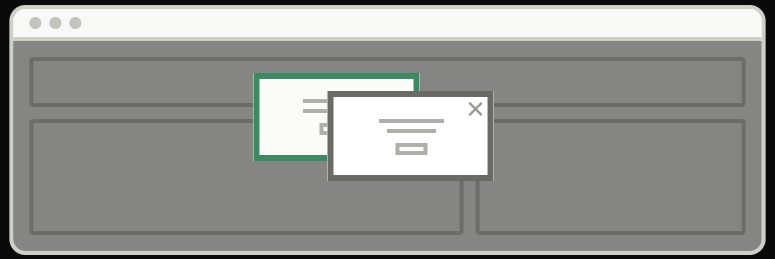
```
const { open } = useModal({
  component: VueFinalDialog,
})
open()
```

component 可以是

- VueFinalModal — opened
- VueFinalDialog — opened
- 用 VueFinalModal 做的 HOC
  - Dialog
  - NavDrawer
  - BottomSheet
  - Spinner
- 你自己的 Modal 元件

push →

```
<ModalsContainer />          modals: 2
1 VueFinalModal
2 VueFinalDialog
```



— 07 · V5 概覽

一個入口，兩種 modal

```
const { open } = useModal({
  component: Spinner,
})
open()
```

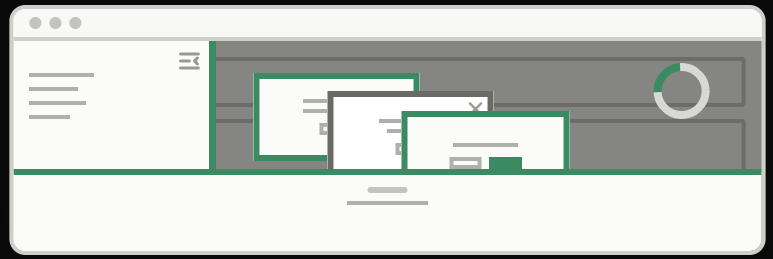
component 可以是

- VueFinalModal — opened
- VueFinalDialog — opened
- 用 VueFinalModal 做的 HOC — opened ×4
  - Dialog
  - NavDrawer
  - BottomSheet
  - Spinner
- 你自己的 Modal 元件

push →

<ModalsContainer /> modals: 6

1	VueFinalModal	
2	VueFinalDialog	
3	Dialog	VueFinalModal
4	NavDrawer	VueFinalModal
5	BottomSheet	VueFinalModal
6	Spinner	VueFinalModal



— 07 · V5 概覽

一個入口，兩種 modal

```
const { open } = useModal({
  component: MyModal,
})
open()
```

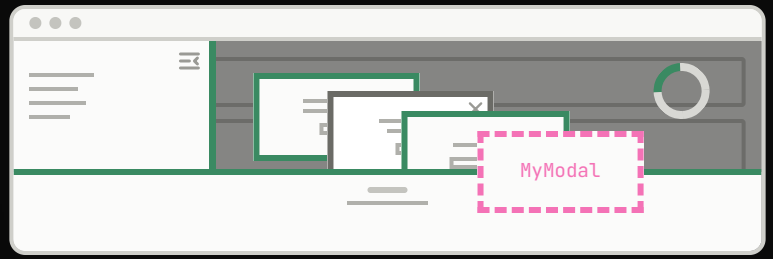
component 可以是

- VueFinalModal — opened
- VueFinalDialog — opened
- 用 VueFinalModal 做的 HOC — opened ×4
  - Dialog
  - NavDrawer
  - BottomSheet
  - Spinner
- 你自己的 Modal 元件 — opened

push →

<ModalsContainer /> modals: 7

1. VueFinalModal
2. VueFinalDialog
3. Dialog — VueFinalModal
4. NavDrawer — VueFinalModal
5. BottomSheet — VueFinalModal
6. Spinner — VueFinalModal
7. MyModal



07 · V5 概覽

兩種 modal，一套契約

VueFinalDialog 包的是原生 <dialog>，VueFinalModal 是 VFM 自己實作的 modal

原生 dialog 與 VueFinalModal 一樣的

- Esc 關閉
- 點外面關閉
- role · focus
- before 事件可擋
- open() 一定有結果

07 · V5 概覽

兩種 modal，一套契約

VueFinalDialog 包的是原生 <dialog>，VueFinalModal 是 VFM 自己實作的 modal

原生 dialog 與 VueFinalModal 一樣的

- Esc 關閉
- 點外面關閉
- role · focus
- before 事件可擋
- open() 一定有結果

原生 dialog 做不到，只有 VueFinalModal 做得到

- 背景可互動
- 關在容器裡
- SSR 首屏就開著
- 只留一層遮罩
- 手勢關閉

07 · V5 概覽

兩種 modal，一套契約

VueFinalDialog 包的是原生 <dialog>，VueFinalModal 是 VFM 自己實作的 modal

原生 dialog 與 VueFinalModal 一樣的

- Esc 關閉
- 點外面關閉
- role · focus
- before 事件可擋
- open() 一定有結果

原生 dialog 做不到，只有 VueFinalModal 做得到

- 背景可互動
- 關在容器裡
- SSR 首屏就開著
- 只留一層遮罩
- 手勢關閉

兩邊都接 useModal，只換 component 那一行。

— 08 · vue-use-template

VFM 的動態渲染層

`useModal()` 底層是 `useTemplate()`

`<ModalsContainer />` 底層是 `<TemplateProvider />`

```ts
// v4
const modal = useModal({
  component: ModalConfirm,
  attrs: { title: 'Hello World!' },
})

modal.patchOptions({
  attrs: { title: 'Hello Vue!' },
})
```

改資料，畫面跟著變

v4	patchOptions，手動同步一次
getter	函式裡讀 ref，改 ref 就同步
ref	整包是 ref，改 .value
computed	算出來的也行，例如 i18n
reactive	直接改屬性
props	props、emits 同一套，plain object 也行
slots	巢狀 template 的 props 一樣會動
型別	全從 component 推出來，錯的當場紅

— 08 · vue-use-template

VFM 的動態渲染層

`useModal()` 底層是 `useTemplate()`

`<ModalsContainer />` 底層是 `<TemplateProvider />`

```ts
// v5 · getter
const title = ref('Hello World!')

const { open } = useModal({
  component: ModalConfirm,
  attrs: () => ({ title: title.value }),
})

title.value = 'Hello Vue!'
```

改資料，畫面跟著變

v4	patchOptions，手動同步一次
getter	函式裡讀 ref，改 ref 就同步
ref	整包是 ref，改 .value
computed	算出來的也行，例如 i18n
reactive	直接改屬性
props	props、emits 同一套，plain object 也行
slots	巢狀 template 的 props 一樣會動
型別	全從 component 推出來，錯的當場紅

— 08 · vue-use-template

VFM 的動態渲染層

`useModal()` 底層是 `useTemplate()`

`<ModalsContainer />` 底層是 `<TemplateProvider />`

```ts
// v5 · ref
const attrs = ref({ title: 'Hello World!' })

const { open } = useModal({
  component: ModalConfirm,
  attrs,
})

attrs.value.title = 'Hello Vue!'
```

改資料，畫面跟著變

v4	patchOptions，手動同步一次
getter	函式裡讀 ref，改 ref 就同步
ref	整包是 ref，改 .value
computed	算出來的也行，例如 i18n
reactive	直接改屬性
props	props、emits 同一套，plain object 也行
slots	巢狀 template 的 props 一樣會動
型別	全從 component 推出來，錯的當場紅

— 08 · vue-use-template

VFM 的動態渲染層

`useModal()` 底層是 `useTemplate()`

`<ModalsContainer />` 底層是 `<TemplateProvider />`

```ts
// v5 · computed
const lang = ref('en')

const { open } = useModal({
  component: ModalConfirm,
  attrs: computed(() => ({ title: t(lang.value) })),
})

lang.value = 'zh-TW'
```

改資料，畫面跟著變

v4	patchOptions，手動同步一次
getter	函式裡讀 ref，改 ref 就同步
ref	整包是 ref，改 .value
computed	算出來的也行，例如 i18n
reactive	直接改屬性
props	props、emits 同一套，plain object 也行
slots	巢狀 template 的 props 一樣會動
型別	全從 component 推出來，錯的當場紅

— 08 · vue-use-template

VFM 的動態渲染層

`useModal()` 底層是 `useTemplate()`

`<ModalsContainer />` 底層是 `<TemplateProvider />`

```ts
// v5 · reactive
const attrs = reactive({ title: 'Hello World!' })

const { open } = useModal({
  component: ModalConfirm,
  attrs,
})

attrs.title = 'Hello Vue!'
```

改資料，畫面跟著變

v4	patchOptions，手動同步一次
getter	函式裡讀 ref，改 ref 就同步
ref	整包是 ref，改 .value
computed	算出來的也行，例如 i18n
reactive	直接改屬性
props	props、emits 同一套，plain object 也行
slots	巢狀 template 的 props 一樣會動
型別	全從 component 推出來，錯的當場紅

— 08 · vue-use-template

VFM 的動態渲染層

`useModal()` 底層是 `useTemplate()`

`<ModalsContainer />` 底層是 `<TemplateProvider />`

```
// v5 · props、emits 同一套
const { open, close } = useModal({
  component: ModalConfirm,
  props: reactive({ title: 'Hello World!' }),
  emits: computed(() => ({
    onConfirm: () => close(),
  })),
})
```

改資料，畫面跟著變

v4	patchOptions，手動同步一次
getter	函式裡讀 ref，改 ref 就同步
ref	整包是 ref，改 .value
computed	算出來的也行，例如 i18n
reactive	直接改屬性
props	**props、emits 同一套，plain object 也行**
slots	巢狀 template 的 props 一樣會動
型別	全從 component 推出來，錯的當場紅

— 08 · vue-use-template

VFM 的動態渲染層

`useModal()` 底層是 `useTemplate()`

`<ModalsContainer />` 底層是 `<TemplateProvider />`

```
// v5 · slots 裡的巢狀 template 也一樣
const { open } = useModal({
  component: ModalConfirm,
  attrs: () => ({ title: title.value }),
  slots: {
    default: defineTemplate({
      component: Hint,
      props: () => ({ text: hint.value }),
    }),
  },
})
```

改資料，畫面跟著變

v4	patchOptions，手動同步一次
getter	函式裡讀 ref，改 ref 就同步
ref	整包是 ref，改 .value
computed	算出來的也行，例如 i18n
reactive	直接改屬性
props	props、emits 同一套，plain object 也行
slots	巢狀 template 的 props 一樣會動
型別	全從 component 推出來，錯的當場紅

— 08 · vue-use-template

VFM 的動態渲染層

`useModal()` 底層是 `useTemplate()`

`<ModalsContainer />` 底層是 `<TemplateProvider />`

```ts
// v5 · 型別跟著 component 走
const { open, close } = useModal({
  component: ModalConfirm,
  attrs: () => ({ title: 1 }), // 紅：要 string
  emits: {
    onConfirm: value => close(), // value 是 number
  },
  slots: { header: Hint }, // 紅：沒有 header slot
})
```

ModalConfirm.vue

```ts
defineProps<{ title: string, count?: number }>()
defineEmits<{ confirm: [value: number], cancel: [] }>()
<slot /> <slot name="footer" />
```

改資料，畫面跟著變

v4	patchOptions，手動同步一次
getter	函式裡讀 ref，改 ref 就同步
ref	整包是 ref，改 .value
computed	算出來的也行，例如 i18n
reactive	直接改屬性
props	props、emits 同一套，plain object 也行
slots	巢狀 template 的 props 一樣會動
型別	全從 component 推出來，錯的當場紅

09 · 原生 dialog 行為比較

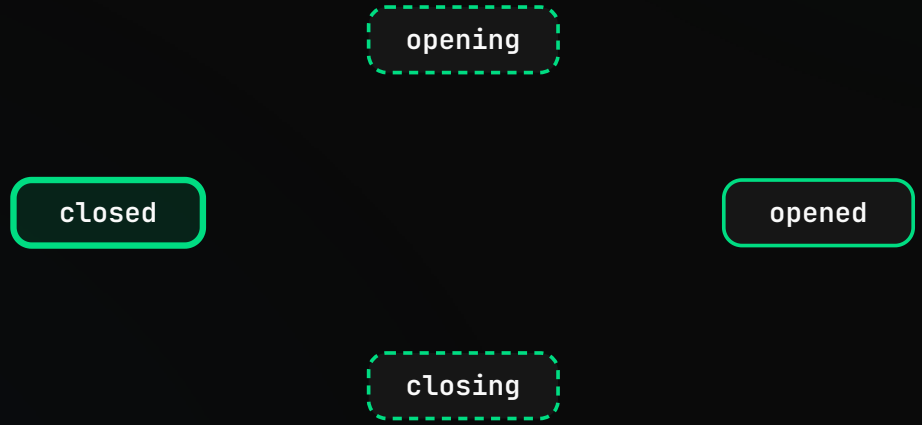
開跟關的每一條路，都有結果

原生 <dialog> `showModal()` / `close()` 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點

09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

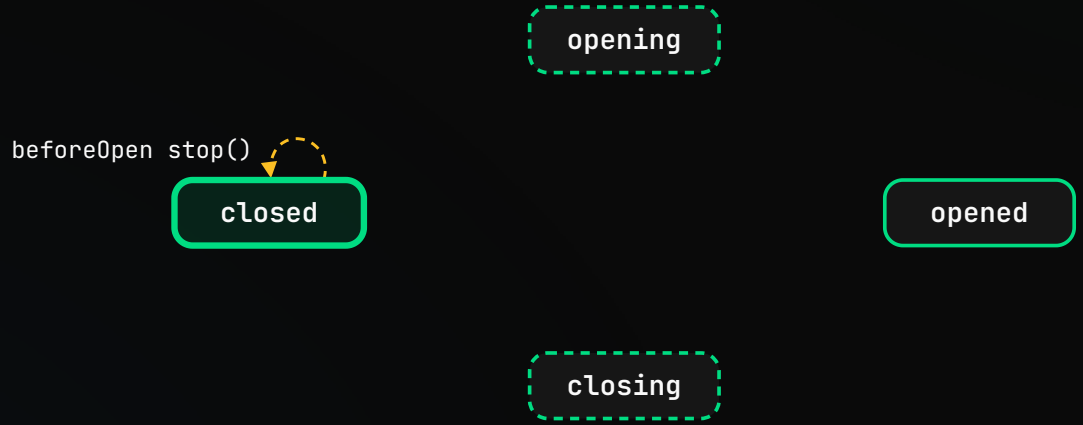
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

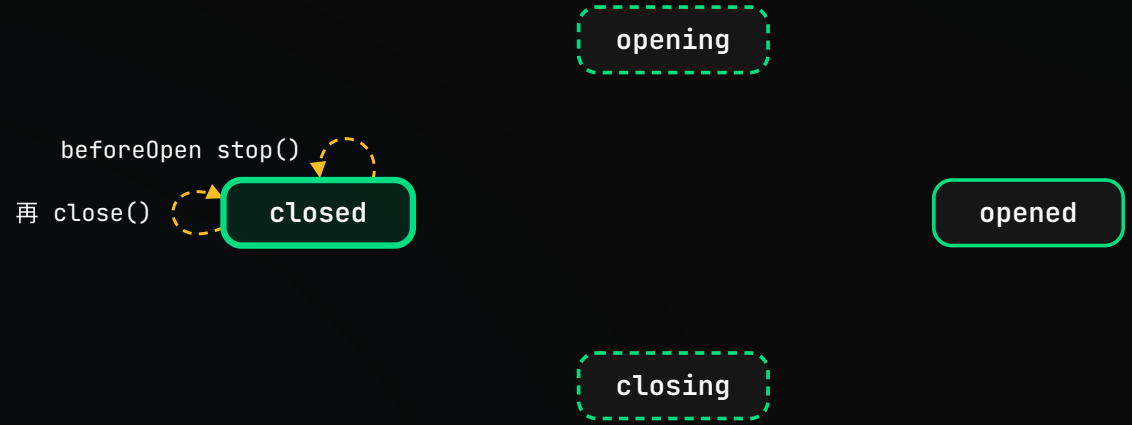
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

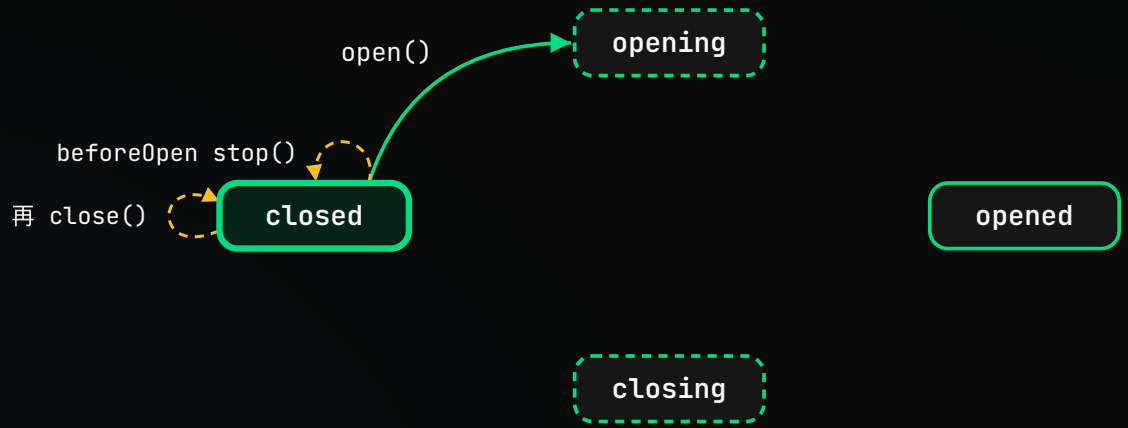
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

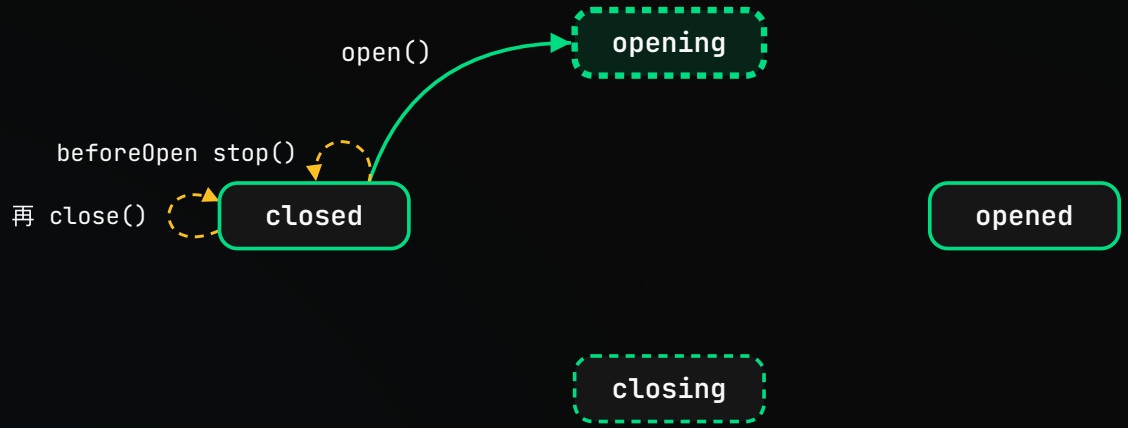
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

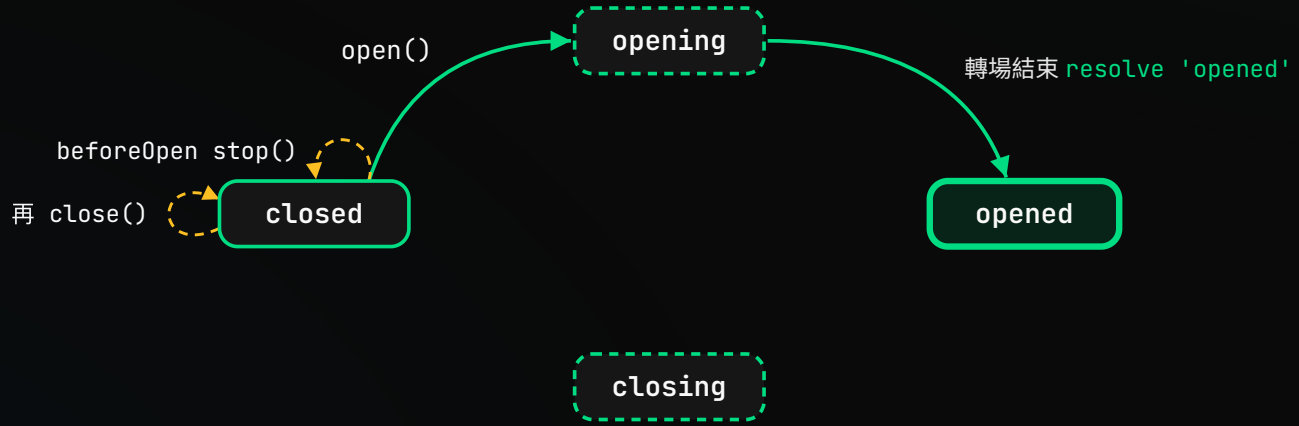
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

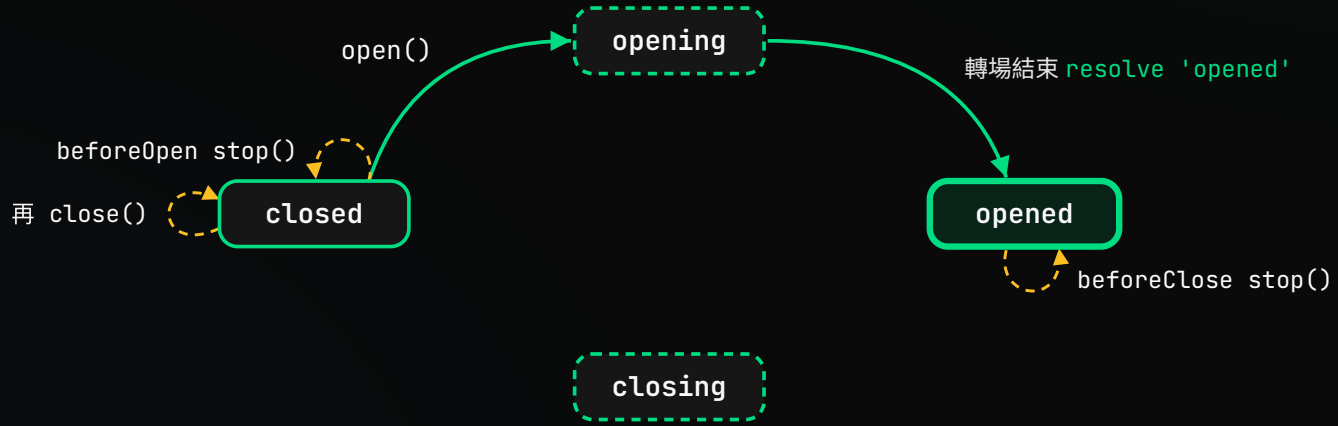
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

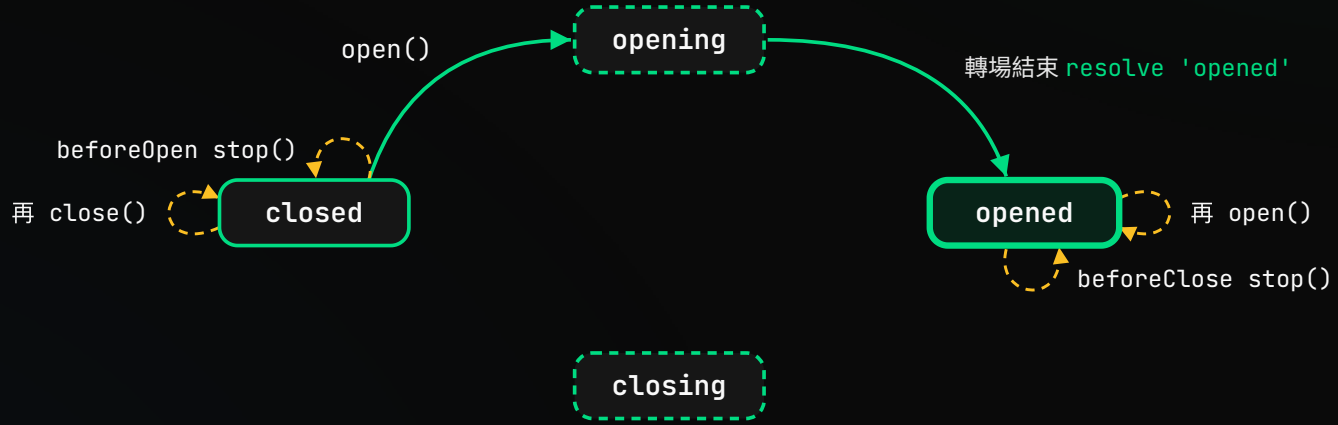
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

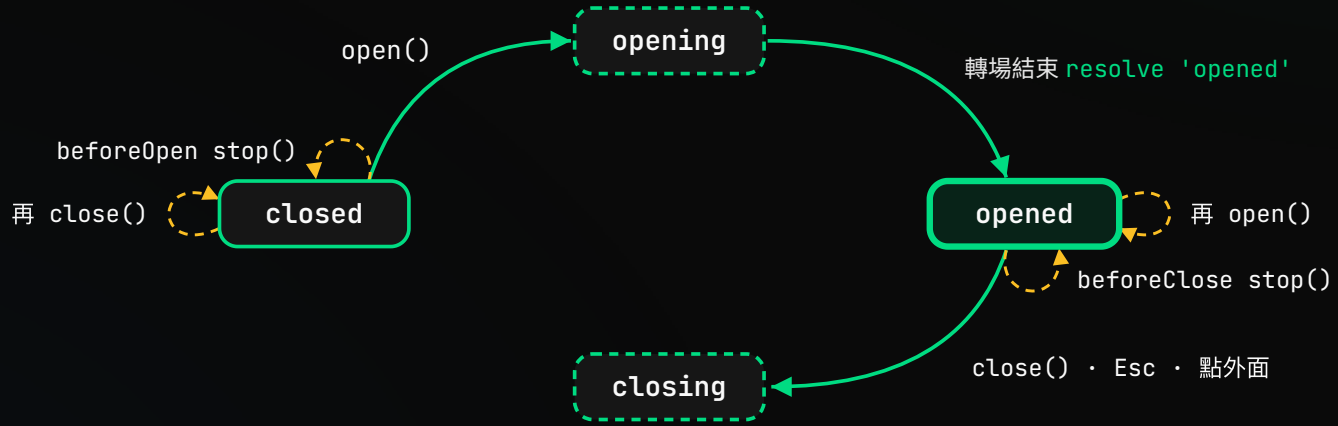
原生 <dialog> `showModal()` / `close()` 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



— 09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

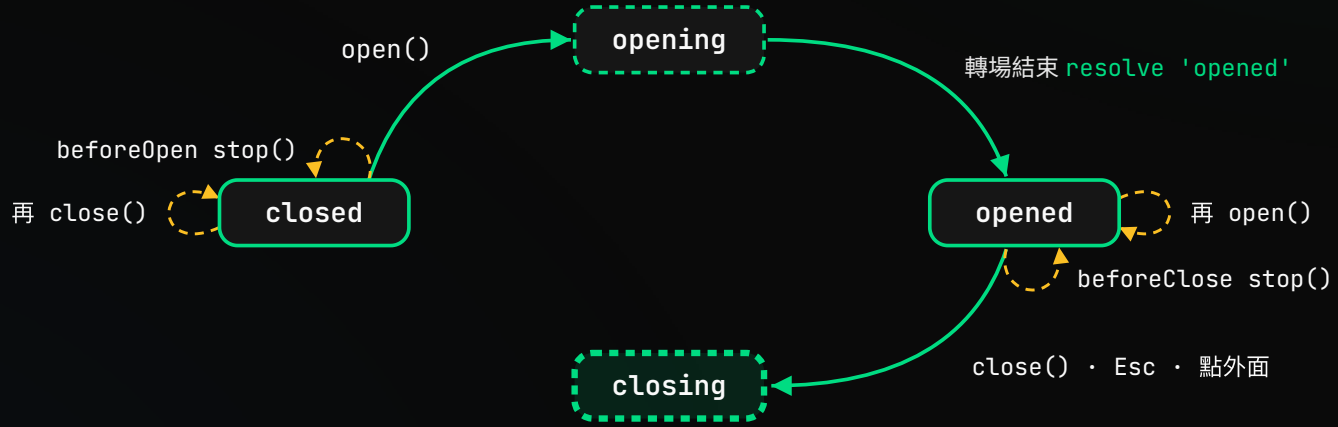
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

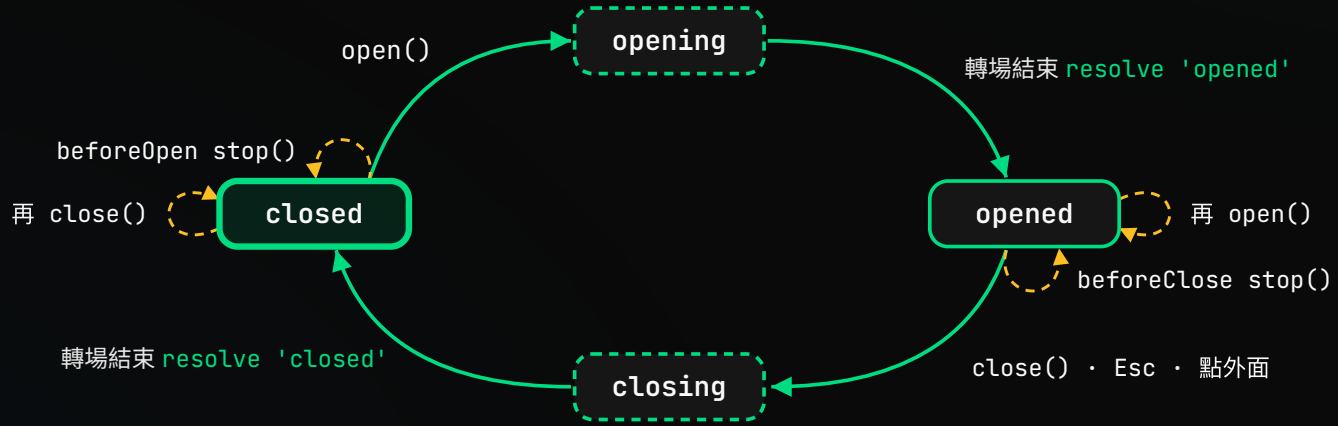
原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



— 09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

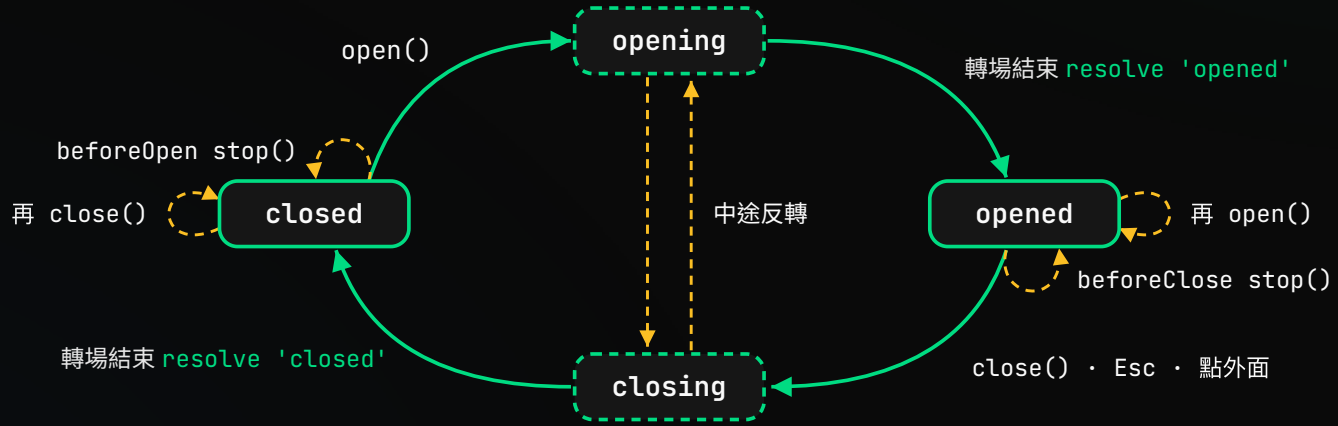
原生 <dialog> `showModal()` / `close()` 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



09 · 原生 dialog 行為比較

開跟關的每一條路，都有結果

原生 <dialog> showModal() / close() 是同步的，呼叫完就回來，沒有「轉場完成」這個時間點



— 09 · 原生 dialog 行為比較

選取文字時，不要把 modal 關掉

原生 <dialog>

預設點遮罩不會關；`closedby="any"` 才會，判定交給瀏覽器，Safari 未支援

09 · 原生 dialog 行為比較

選取文字時，不要把 modal 關掉

原生 <dialog>

預設點遮罩不會關；`closedby="any"` 才會，判定交給瀏覽器，Safari 未支援

```
function onMousedown(e: MouseEvent) {
  lastMousedownEl = e.target
}

function onMouseupRoot() {
  if (lastMousedownEl !== vfmRootEl.value) return
  // 從遮罩上按下、也在遮罩上放開，才算點外面
}
```



09 · 原生 dialog 行為比較

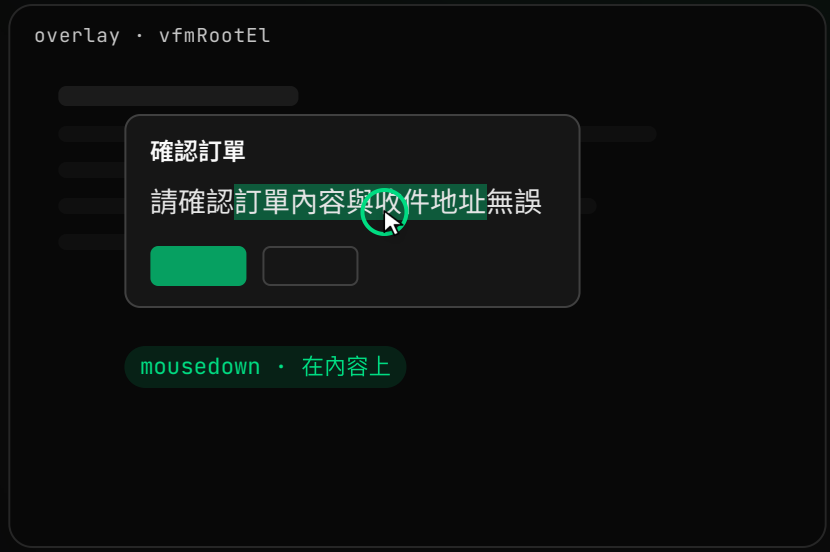
選取文字時，不要把 modal 關掉

原生 <dialog>

預設點遮罩不會關；`closedby="any"` 才會，判定交給瀏覽器，Safari 未支援

```
function onMousedown(e: MouseEvent) {
  lastMousedownEl = e.target
}

function onMouseupRoot() {
  if (lastMousedownEl !== vfmRootEl.value) return
  // 從遮罩上按下、也在遮罩上放開，才算點外面
}
```



09 · 原生 dialog 行為比較

選取文字時，不要把 modal 關掉

原生 <dialog>

預設點遮罩不會關；`closedby="any"` 才會，判定交給瀏覽器，Safari 未支援

```
function onMousedown(e: MouseEvent) {
  lastMousedownEl = e.target
}

function onMouseupRoot() {
  if (lastMousedownEl !== vfmRootEl.value) return
  // 從遮罩上按下、也在遮罩上放開，才算點外面
}
```



不算點外面，modal 留著

09 · 原生 dialog 行為比較

疊三層 modal，畫面不該越來越暗

— 09 · 原生 dialog 行為比較

疊三層 modal，畫面不該越來越暗

原生：每個 dialog 各自一個 ::backdrop



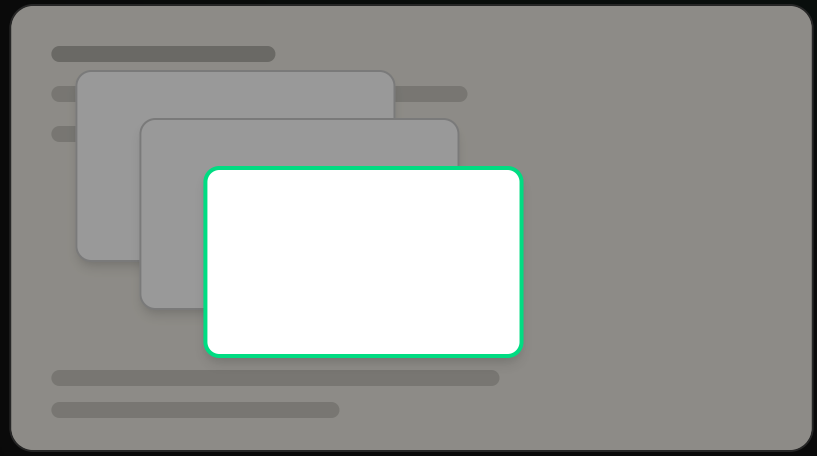
— 09 · 原生 dialog 行為比較

疊三層 modal，畫面不該越來越暗

原生：每個 dialog 各自一個 ::backdrop



VFM：`overlayBehavior: 'auto'`



示意圖；`overlayBehavior: 'persist'` 則保留每一層的遮罩

09 · 原生 dialog 行為比較

捲動鎖與焦點

捲動鎖

原生 dialog 不鎖捲動，背景照樣能捲；VFM 用重寫的 scroll-lock 鎖住，Nuxt 環境也安全

焦點

原生 dialog 會把焦點關在裡面；VueFinalModal 用 focus-trap 做到一樣的事



捲動鎖

body 被鎖了幾次

0

FOCUS TRAP 堆疊

焦點在

頁面上的「開啟 A」

09 · 原生 dialog 行為比較

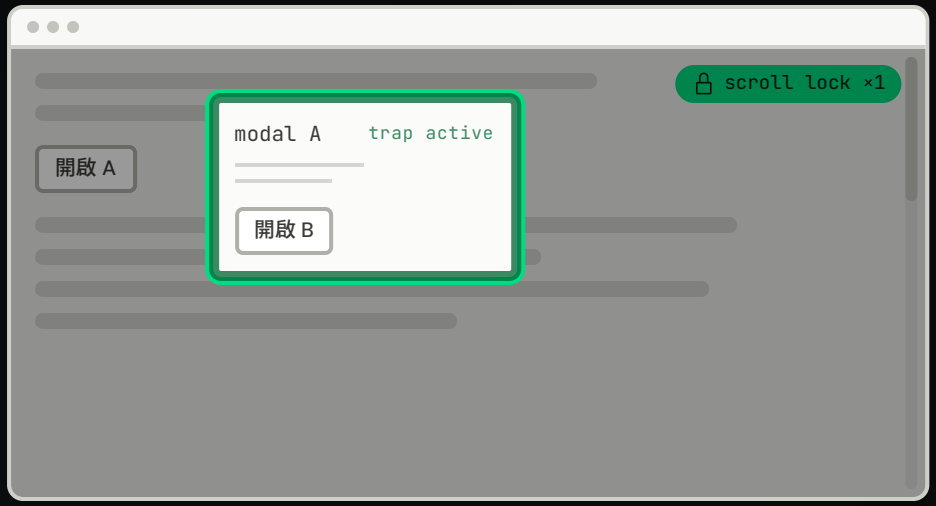
捲動鎖與焦點

捲動鎖

原生 dialog 不鎖捲動，背景照樣能捲；VFM 用重寫的 scroll-lock 鎖住，Nuxt 環境也安全

焦點

原生 dialog 會把焦點關在裡面；VueFinalModal 用 focus-trap 做到一樣的事



捲動鎖

body 被鎖了幾次

1

FOCUS TRAP 堆疊

trap A active

焦點在

A 的內容區

09 · 原生 dialog 行為比較

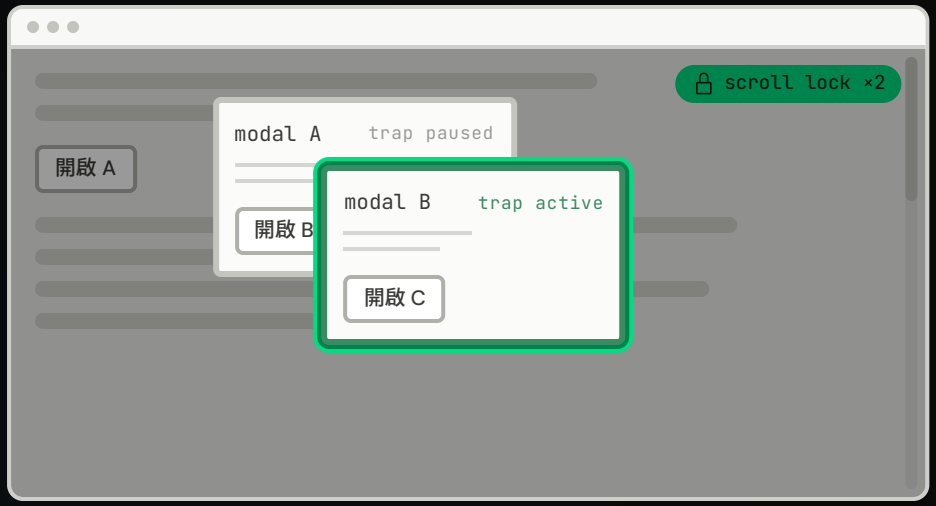
捲動鎖與焦點

捲動鎖

原生 dialog 不鎖捲動，背景照樣能捲；VFM 用重寫的 scroll-lock 鎖住，Nuxt 環境也安全

焦點

原生 dialog 會把焦點關在裡面；VueFinalModal 用 focus-trap 做到一樣的事



捲動鎖

body 被鎖了幾次

2

FOCUS TRAP 堆疊

trap B active

trap A paused

焦點在

B 的內容區

— 09 · 原生 dialog 行為比較

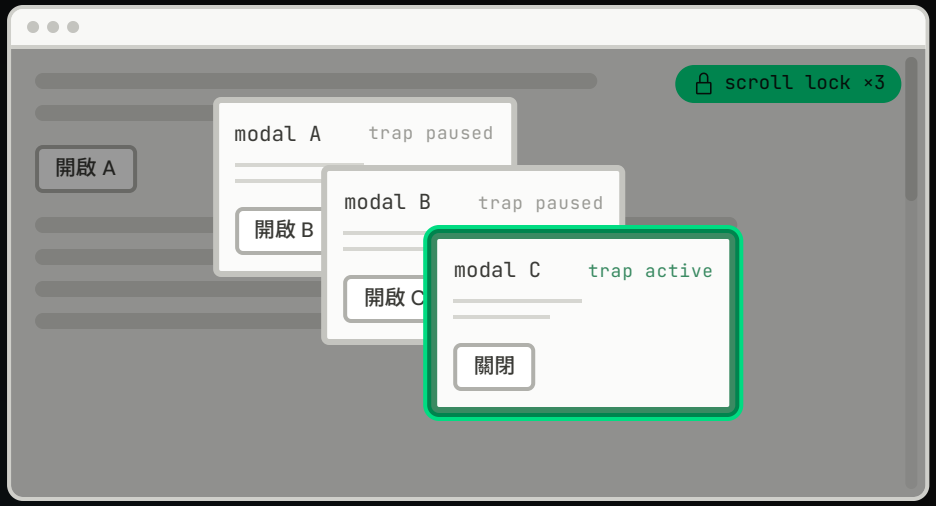
捲動鎖與焦點

捲動鎖

原生 dialog 不鎖捲動，背景照樣能捲；VFM 用重寫的 scroll-lock 鎖住，Nuxt 環境也安全

焦點

原生 dialog 會把焦點關在裡面；VueFinalModal 用 focus-trap 做到一樣的事



捲動鎖

body 被鎖了幾次

3

FOCUS TRAP 堆疊

trap C	active
trap B	paused
trap A	paused

焦點在

C 的內容區

— 09 · 原生 dialog 行為比較

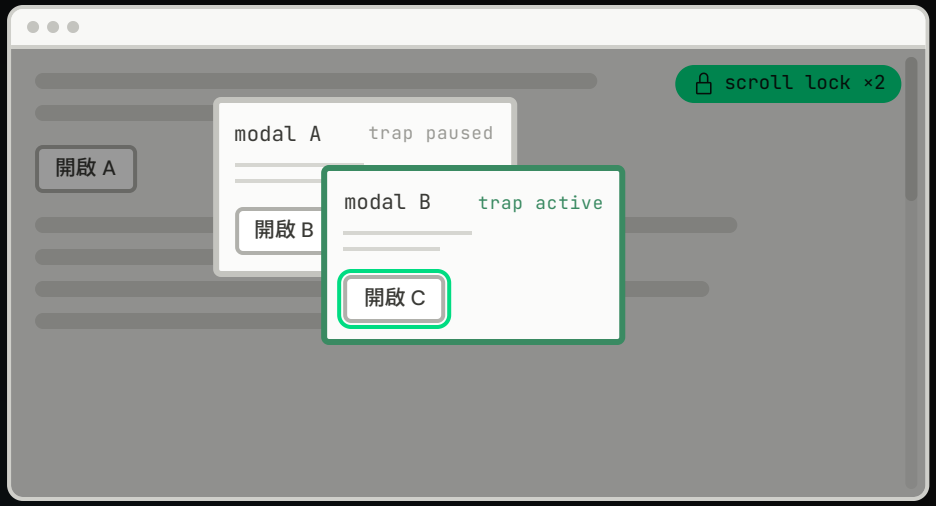
捲動鎖與焦點

捲動鎖

原生 dialog 不鎖捲動，背景照樣能捲；VFM 用重寫的 scroll-lock 鎖住，Nuxt 環境也安全

焦點

原生 dialog 會把焦點關在裡面；VueFinalModal 用 focus-trap 做到一樣的事



捲動鎖

body 被鎖了幾次

2

FOCUS TRAP 堆疊

trap B active

trap A paused

焦點在

回到 B 的「開啟 C」

09 · 原生 dialog 行為比較

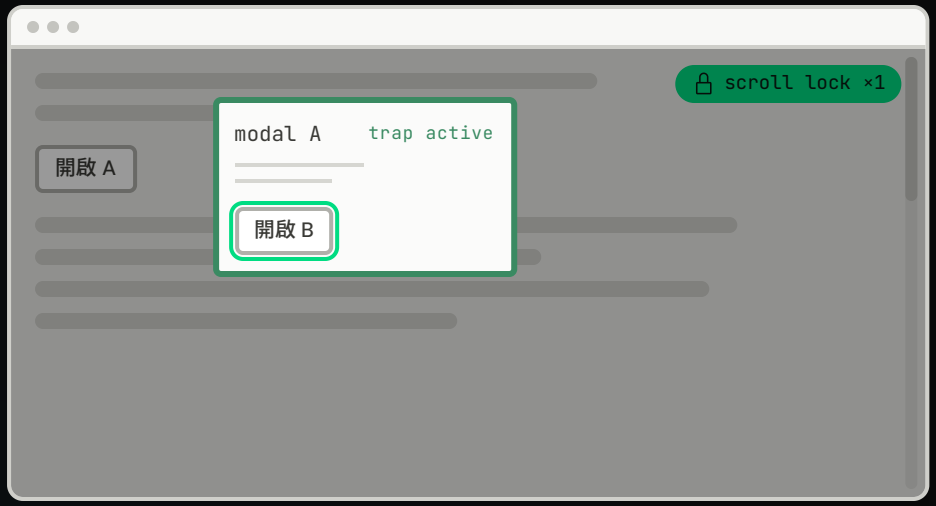
捲動鎖與焦點

捲動鎖

原生 dialog 不鎖捲動，背景照樣能捲；VFM 用重寫的 scroll-lock 鎖住，Nuxt 環境也安全

焦點

原生 dialog 會把焦點關在裡面；VueFinalModal 用 focus-trap 做到一樣的事



捲動鎖

body 被鎖了幾次

1

FOCUS TRAP 堆疊

trap A active

焦點在

回到 A 的「開啟 B」

09 · 原生 dialog 行為比較

捲動鎖與焦點

捲動鎖

原生 dialog 不鎖捲動，背景照樣能捲；VFM 用重寫的 scroll-lock 鎖住，Nuxt 環境也安全

焦點

原生 dialog 會把焦點關在裡面；VueFinalModal 用 focus-trap 做到一樣的事



捲動鎖

body 被鎖了幾次

0

FOCUS TRAP 堆疊

焦點在

回到頁面上的「開啟 A」

一個套件，附 Nuxt module

一個套件，附 Nuxt module

V4 · 另一個套件

```
// pnpm add @vue-final-modal/nuxt
export default defineNuxtConfig({
  modules: ['@vue-final-modal/nuxt'],
})
```

一個套件，附 Nuxt module

V4 · 另一個套件

```
// pnpm add @vue-final-modal/nuxt
export default defineNuxtConfig({
  modules: ['@vue-final-modal/nuxt'],
})
```



V5 · 隨主套件發布

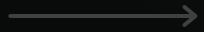
```
export default defineNuxtConfig({
  modules: ['vue-final-modal/nuxt'],
})
```

10 · SSR 與 NUXT

server browser

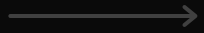
setup 內

```
<script setup lang="ts">
const { open } = useModal({
  component: Confirm,
})
open()
</script>
```



setup 外

```
// 任何元件之外，例如 plugin
const { open } = useModal({
  component: Confirm,
})
open()
```

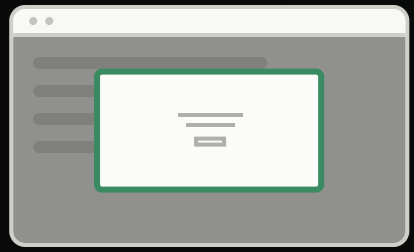


server

browser

setup 內

```
<script setup lang="ts">
const { open } = useModal({
  component: Confirm,
})
open()
</script>
```



寫進 HTML，帶 z-index

setup 外

```
// 任何元件之外，例如 plugin
const { open } = useModal({
  component: Confirm,
})
open()
```

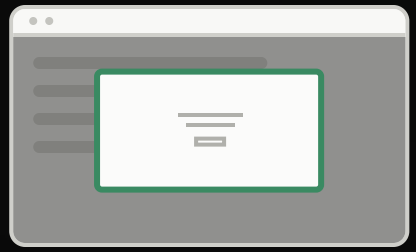
setup 內

```
<script setup lang="ts">
const { open } = useModal({
  component: Confirm,
})
open()
</script>
```

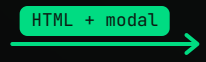
setup 外

```
// 任何元件之外，例如 plugin
const { open } = useModal({
  component: Confirm,
})
open()
```

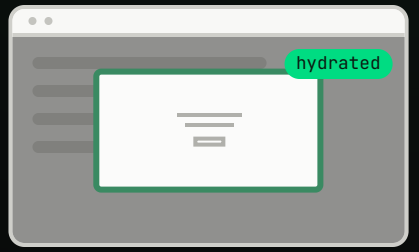
server



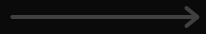
寫進 HTML，帶 z-index



browser



立刻出現，水合完成就能操作



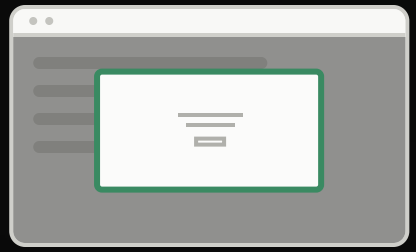
setup 內

```
<script setup lang="ts">
const { open } = useModal({
  component: Confirm,
})
open()
</script>
```

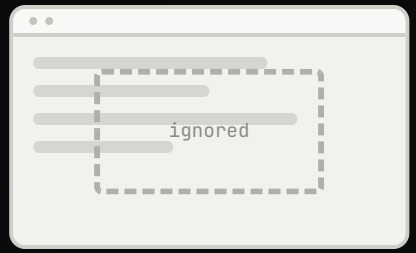
setup 外

```
// 任何元件之外，例如 plugin
const { open } = useModal({
  component: Confirm,
})
open()
```

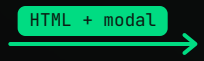
server

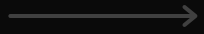


寫進 HTML，帶 z-index

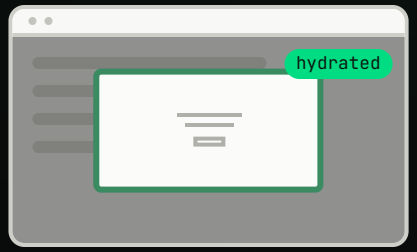


分不出 request，跳過





browser



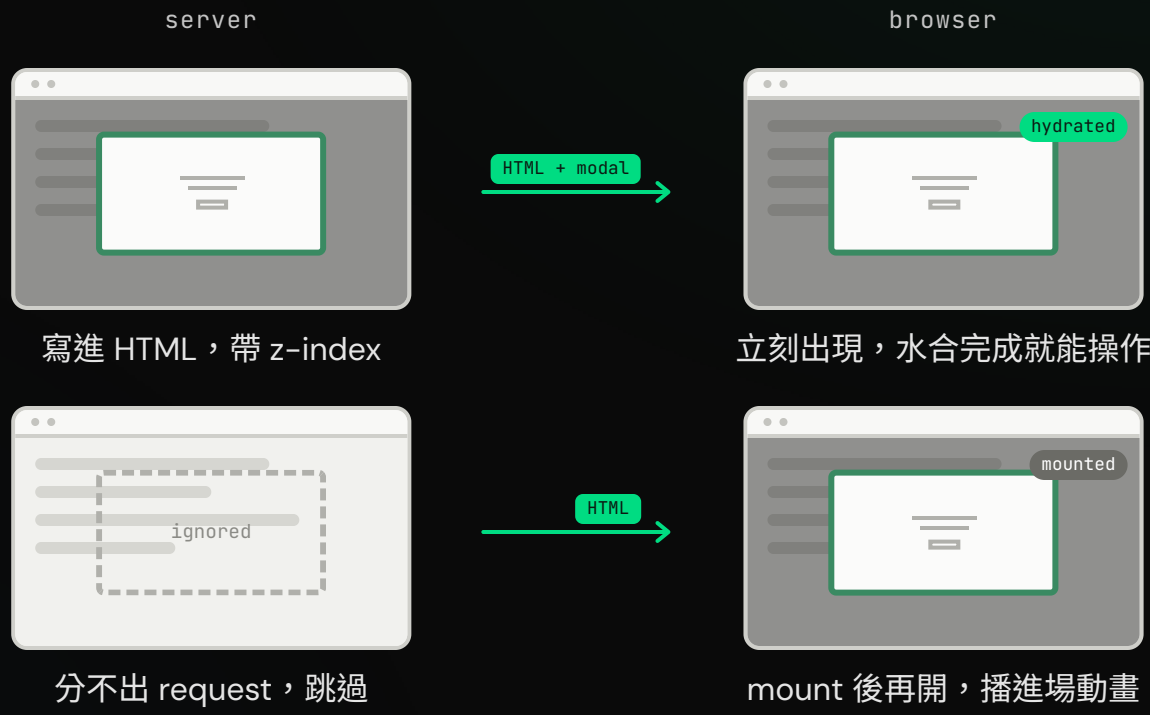
立刻出現，水合完成就能操作

setup 內

```
<script setup lang="ts">
const { open } = useModal({
  component: Confirm,
})
open()
</script>
```

setup 外

```
// 任何元件之外，例如 plugin
const { open } = useModal({
  component: Confirm,
})
open()
```



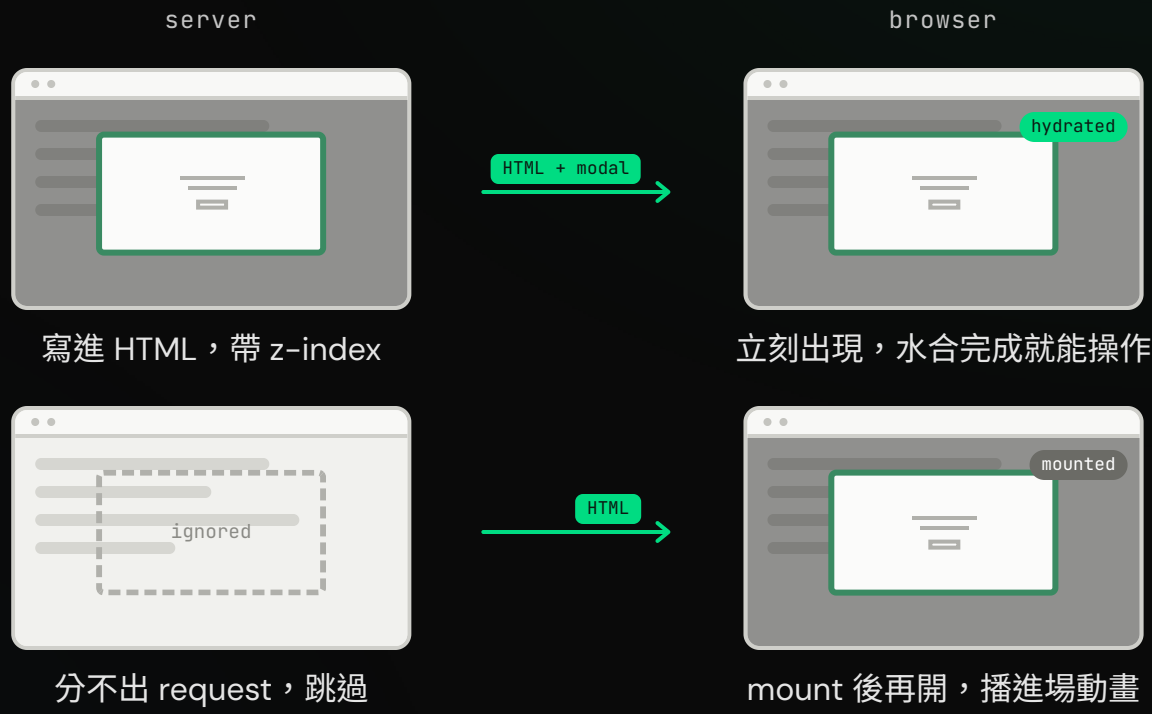
10 · SSR 與 NUXT

setup 內

```
<script setup lang="ts">
const { open } = useModal({
  component: Confirm,
})
open()
</script>
```

setup 外

```
// 任何元件之外，例如 plugin
const { open } = useModal({
  component: Confirm,
})
open()
```



兩條路 server 與 browser 的輸出都一致 `沒有 hydration mismatch` v4 的 `server render memory leak` 也一起解掉



這次真的是 Final 版了



v5.0.0

現在發布

結語

從 v4 遷移

結語

從 v4 遷移

```
❯ claude
```

結語

從 v4 遷移

```
❯ claude

> 幫我把 vue-final-modal 升級到 v5
```

結語

從 v4 遷移

```
❯ claude

> 幫我把 vue-final-modal 升級到 v5

● 正在讀 vue-final-modal.org 的遷移指南……
```

Migration Guide

v5.vue-final-modal.org

Contributors



💖 github.com/vue-final/vue-final-modal

感謝

投影片稍後將在 github.com/hunterliu1003/talks 開放

💖 感謝 Anthony Fu，投影片由 Slidev 強力驅動！

💖 感謝 sxzz，tsdown 的介紹參考自 github.com/sxzz/talks



2026-10-17